

DESIGNING A SKILLED SOCCER TEAM FOR ROBOCUP: EXPLORING SKILL-SET-PRIMITIVES THROUGH REINFORCEMENT LEARNING

Miguel Abreu & Luis Paulo Reis

LIACC/LASI/FEUP, Artificial Intelligence
and Computer Science Laboratory,
Faculty of Engineering, University of Porto,
Porto, Portugal
{m.abreu, lpreis}@fe.up.pt

Nuno Lau

IEETA/LASI/DETI, Institute of Electronics
and Informatics Engineering of Aveiro,
Department of Electronics Telecommunications
and Informatics, University of Aveiro,
Aveiro, Portugal
nunolau@ua.pt

ABSTRACT

The RoboCup 3D Soccer Simulation League serves as a competitive platform for showcasing innovation in autonomous humanoid robot agents through simulated soccer matches. Our team, FC Portugal, developed a new codebase from scratch in Python after RoboCup 2021. The team’s performance relies on a set of skills centered around novel unifying primitives and a custom, symmetry-extended version of the Proximal Policy Optimization algorithm. Our methods have been thoroughly tested in official RoboCup matches, where FC Portugal has won the last two main competitions, in 2022 and 2023. This paper presents our training framework, as well as a timeline of skills developed using our skill-set-primitives, which considerably improve the sample efficiency and stability of skills, and motivate seamless transitions. We start with a significantly fast sprint-kick developed in 2021 and progress to the most recent skill set, including a multi-purpose omnidirectional walk, a dribble with unprecedented ball control, a solid kick, and a push skill. The push addresses low-level collision scenarios and high-level strategies to increase ball possession. We address the resource-intensive nature of this task through an innovative multi-agent learning approach. Finally, we release the team’s codebase to the RoboCup community, providing other teams with a robust and modern foundation upon which they can build new features.

1 INTRODUCTION

RoboCup is an international competition standing at the forefront of robotics research and innovation. It features four main leagues: @Home for domestic robot assistance, Industrial for logistics and warehousing systems, Rescue for disaster strategies and rescue operations, and Soccer, the largest league, focusing on competitive robotics.

The primary objective of this project is to enable autonomous humanoid teams to win against human soccer champions by 2050 [31]. Achieving this goal requires addressing the disparities between human and robotic capabilities in decision-making, coordination, and skill execution.

Our focus lies on the 3D Soccer Simulation League (3DSSL) — a simulated soccer competition involving two teams of autonomous humanoid robots. Established in 2004, the league adopted the NAO robot model in 2008. Initially, matches were played on a 12 m by 8 m field with teams of three robots [9]. In 2012, the league expanded to its current dimensions of 30 m by 20 m, allowing for full teams of 11 players.

Some of the 3DSSL’s main challenges and research efforts have been directed toward skill optimization, real-time decision-making, and cooperative multi-agent strategies [16]. Robots must execute fundamental tasks like locomotion and kicking while dynamically adapting to game scenarios involving role allocation and team coordination. Additionally, addressing these issues also requires overcoming challenges such as sensor noise, localization under uncertainty, and collision-free path

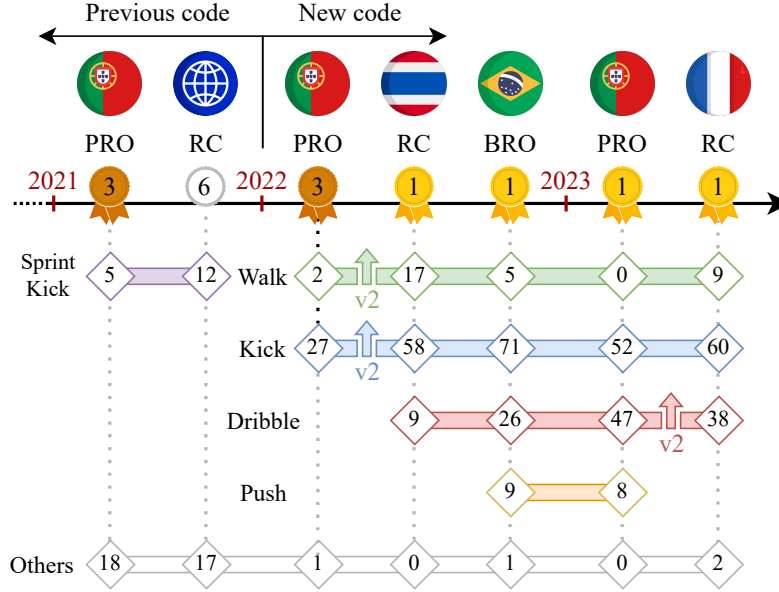


Figure 1: Timeline of skills (based on skill-set-primitives) that were used in official competitions. Each rhombus indicates the tally of goals directly attributed to each skill in the respective competition

planning. Some of these concerns are also applicable to real-world robotics, giving the research in this area broader implications beyond robotic soccer or RoboCup.

This paper introduces a novel approach to develop a cohesive structure of skills that works in harmony at different levels of abstraction — from motor control to strategy — using primitives that represent the commonalities of a skill set. All the methods detailed in this paper have been validated through participation in several official competitions, serving as the central pillar of our team, FC Portugal.

Figure 1 depicts a timeline of skills developed with skill-set-primitives. In 2020, we developed the Sprint-Kick¹, a behavior that sprinted toward the ball during kickoff, resulting in 22% of our team’s goals in the Portuguese Robotics Open (PRO) 2021. In RoboCup (RC) 2021 (virtual event), the kickoff was refined, accounting for 38% of all our goals². We secured 3rd and 6th place, respectively, having good individual skills but lacking cohesion.

Post RC 2021, we rewrote the codebase from scratch in Python with three goals: reliability (minimal skill failure and collision resistance), performance (ball possession, precision, and speed), and cohesion (harmony, fast integration, and easy transitions). A new agent debuted in PRO 2022 with only 3 skills: Get Up, Omnidirectional Walk, and Kick. The walk was developed on top of a new primitive, and the kick was fixed at around 9 m, scoring 27 goals. The walk scored twice, and the remaining goal was an own goal in our favor. The team ranked 3rd, but would already win consistently against our old codebase.

For RC 2022 (Thailand), the walk and kick were upgraded, with the latter now offering distance control and a fixed long option. A secret weapon was unveiled later in the competition — a pioneer and very efficient close-control dribble. The new skills along with a hand-tuned strategy led FC Portugal to its second 3DSSL victory since 2006.

Creating strategies is simpler when players can move freely to strategic positions, but it gets complex when collisions are inevitable. The Push skill was introduced to handle close encounters by using reinforcement learning for both motor control and strategy in a multi-agent environment. This configuration secured two first places in the Brazilian Robotics Open (BRO) in 2022 and PRO 2023.

¹First Sprint-Kick demonstration in 2020 <https://youtu.be/Yy1yCM5hwZI>

²Successful kickoffs in RoboCup 2021 <https://youtu.be/3MND8RVUPBQ>

The dominant dribbling skill allowed any player, even the goalkeeper, to quickly bypass all opponents and score, while keeping the ball between both feet, making defense nearly impossible without fouling. To improve gameplay, RoboCup 2023 (Bordeaux) introduced new rules to prevent ball-holding³, rendering our Dribble and Push skills unusable. Due to time constraints, only the Dribble was retrained to comply with the new rules. FC Portugal won the main competition for the third time in 2023.

Overall, development has primarily focused on offensive skills, with the goalkeeper serving as a strategic position player without specific skills.

While the landscape of robotic soccer teams leans toward Java or C++ programming languages, our approach leveraged the rapid development capabilities of Python, coupled with C++ modules. In this work, we also release the codebase of our team to the community, providing a foundation upon which other teams can build and innovate. The codebase includes an integrated reinforcement learning gym to facilitate the development of new skills.

In addition to releasing the codebase, this work introduces a novel learning methodology with significant contributions. We employ skill-set-primitives to capture commonalities of a set of actions, promoting seamless transitions and reducing the complexity of the learned model. These primitives allow the policy to be simplified into a shallow neural network with a single hidden layer, which improves sample efficiency and stability, while still retaining excellent performance in learning complex behaviors. In the multi-agent setting, we address the resource-intensive task of learning high-level strategies by using virtual agents and rendering only the physical robots of the closest player to the ball on each team. Furthermore, this paper showcases a collection of high-quality skills developed using the proposed learning methodology, with their performance demonstrated in official competitions. Finally, we provide comprehensive information for reproducing these skills.

2 BACKGROUND

Typical reinforcement learning problems can be described as a Markov Decision Process (MDP) – a tuple $\langle \mathcal{S}, \mathcal{A}, \Psi, p, r \rangle$, with a set of states $\mathcal{S}$, a set of actions $\mathcal{A}$, a set of possible state-action pairs $\Psi \subseteq \mathcal{S} \times \mathcal{A}$, a transition function $p(s, a, s') : \Psi \times \mathcal{S} \rightarrow [0, 1]$, and a reward function $r(s, a) : \Psi \rightarrow \mathbb{R}$.

2.1 MDP SYMMETRIES

Model reduction allows the exploitation of redundant or symmetric features. To this end, Ravindran and Barto [48] proposed a mathematical formalism to describe MDP homomorphisms — a transformation that groups equivalent states and actions. An MDP homomorphism h from $M = \langle \mathcal{S}, \mathcal{A}, \Psi, p, r \rangle$ to $\tilde{M} = \langle \tilde{\mathcal{S}}, \tilde{\mathcal{A}}, \tilde{\Psi}, \tilde{p}, \tilde{r} \rangle$ can be defined as a surjection $h : \Psi \rightarrow \tilde{\Psi}$, which is itself defined by a tuple of surjections $\langle f, \{g_s | s \in \mathcal{S}\} \rangle$. Due to symmetry characteristics, for $(s, a) \in \Psi$, $h((s, a)) = (f(s), g_s(a))$ is bijective, where $f : \mathcal{S} \rightarrow \tilde{\mathcal{S}}$ and $g_s : \mathcal{A}_s \rightarrow \tilde{\mathcal{A}}_{f(s)}$, for $s \in \mathcal{S}$, satisfy:

$$p(f(s), g_s(a), f(s')) = p(s, a, s'), \quad \forall s, s' \in \mathcal{S}, a \in \mathcal{A}_s, \quad (1)$$

$$\text{and } r(f(s), g_s(a)) = r(s, a), \quad \forall s \in \mathcal{S}, a \in \mathcal{A}_s. \quad (2)$$

A deduction of these equations can be found in previous work [6].

2.2 LEARNING ALGORITHM

The reinforcement learning algorithm used in this work is the Proximal Policy Optimization (PPO) [50] extended with Proximal Symmetry Loss (PSL)⁴ [6]. PPO was chosen due to its good performance in single-agent and cooperative multi-agent games [66]. PSL leverages the symmetry properties of the NAO robot (sagittal plane), to improve the training sample efficiency and also promote

³Official rules are found at <https://ssim.robocup.org/3d-simulation/3d-rules/>

⁴PPO+PSL algorithm: <https://github.com/m-abr/Adaptive-Symmetry-Learning>

a more human-like behavior, without hindering asymmetric exploration. The extended objective function can be written as:

$$L^{PPO+PSL}(\theta, \omega) = \hat{\mathbb{E}}_t [L_t^C(\theta) - L_t^{VF}(\omega) + cH(\theta) - L_t^{PSL}(\theta, \omega)], \quad (3)$$

$$\text{with } L_t^C(\theta) = \min \left(r_t(\theta) \hat{A}_t, (1 + \text{sgn}(\hat{A}_t) \epsilon) \hat{A}_t \right), \quad (4)$$

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}, \quad (5)$$

where the stochastic policy π_θ is parameterized by θ and the value function by ω . $\pi_{\theta_{old}}$ is a copy of the policy before each update, $\hat{A}_t$ is the estimator of the advantage function, L_t^{VF} is a squared error loss to update the value function, ϵ is a clipping parameter, c is a coefficient and H is the policy’s distribution entropy. The expectation $\hat{\mathbb{E}}_t$ indicates the empirical average over a finite batch of samples. The symmetry loss L^{PSL} is characterized as follows:

$$L^{PSL}(\theta, \omega) = \hat{\mathbb{E}}_t [w_\pi \cdot L_t^\pi(\theta) + w_V \cdot L_t^V(\omega)], \quad (6)$$

$$\text{with } L_t^\pi(\theta) = -\hat{\mathbb{E}}_t [\min(x_t(\theta), 1 + \epsilon)], \quad (7)$$

$$L_t^V(\omega) = \hat{\mathbb{E}}_t [(V_\omega(f(s_t)) - V_t^{\text{targ}})^2], \quad (8)$$

where w_π and w_V are weight vectors, $V_\omega(f(s_t))$ is the symmetric state value, V_t^{targ} is the same target value used by PPO in L^{VF} , and $x_t(\theta)$ is a symmetry probability ratio. An extensive description of this algorithm can be found in previous work [6].

2.3 3D SIMULATION LEAGUE

The RoboCup 3D simulation league uses SimSpark [64] as its physical multi-agent simulator. The league’s environment is a 30 m by 20 m soccer field containing several landmarks that can be used for self-localization: goalposts, corner flags, and lines. Each team consists of 11 humanoid robots modeled after the NAO robot, created by SoftBank Robotics. Agents get internal data (joints, accelerometer, gyroscope, and foot pressure sensors) with a one-step delay every 0.02 s, and visual data (restricted to a 120° vision cone) every 0.04 s (the visual data interval was 0.06 s prior to 2023). Agents can send joint commands every 0.02 s and message teammates every 0.04 s. In addition to the standard NAO model (type 0), there are alternative types with longer limbs (type 1), quicker feet (type 2), wider hips and longest limbs (type 3), and added toes (type 4).

For RoboCup 2023, the Technical Committee has approved a ball-holding rule to prevent the first version of FC Portugal’s dribble, as it was difficult to stop without committing a charging foul. The new rule triggers a foul when a player keeps the ball within 0.12 m of itself for over 5.0 s, with no opponents closer than 0.75 m.

3 RELATED WORK

RoboCup is an international robotics competition that drives AI and robotics research, featuring standardized challenges for robotic teams. Most of the original challenges in 1997 [32] still stand today in the 3D soccer simulation league (3DSSL), including multi-agent collaboration, real-time reasoning, handling dynamic environments, and learning complex tasks.

3.1 LEARNING PARADIGMS

Developing a team involves the creation of an interplay of skills at different levels of abstraction, requiring seamless transitions and coordination between objectives of varying granularity. Hierarchical reinforcement learning is one way of tackling this issue by allowing the decomposition of complex tasks. In light of this, Stone et al. [59] proposed the Layered Learning (LL) paradigm,

where tasks are learned in a bottom-up approach, originally in a sequential way. A lower-level layer (A) is learned and frozen before learning the next layer (B). Posterior formulations include concurrent [63] and overlapping variations [39, 40]. In concurrent LL, (A) is initially learned in isolation, being later refined while simultaneously learning (B). In overlapping LL, there are 3 paradigms that can be succinctly described as: the combination of independently learned layers, a partial version of concurrent LL where (A) is only partially refined, and an iterative refinement of each layer.

Still in the context of learning paradigms, it is important to address the case in which an initial model is built analytically and then refined through optimization algorithms. The most common solutions are optimizing parameters of the analytical formulation, or building a new adjustment layer that modifies the initial model’s output to achieve the target behavior. The difference between the original model and the desired outcome is often called residuals. This decomposition technique is common in robotic environments, particularly when analytical models struggle to accurately represent the complex array of physical interactions. In this context, RL has been employed to refine control models, enhancing performance in manipulation and control tasks [22, 55]. Furthermore, applying residual adjustments has been shown to improve robotic locomotion, enabling more agile and stable movements in unpredictable terrains [26, 33, 65]. These techniques have been widely used, ranging from dexterous hand-object interactions [17] to improving grasping and throwing accuracy [67], highlighting the effectiveness of residual learning in refining robotic behaviors across diverse tasks.

In a cooperative-competitive soccer environment, addressing the game’s multi-agent nature is crucial, even when developing low-level skills. This is particularly important when multiple agents contend for the ball and collisions are likely to occur. Multi-agent reinforcement learning (MARL) presents three major information structures [68]: a centralized setting where a central controller shares information with all agents, a decentralized setting where agents can communicate with each other, and a fully decentralized setting where no explicit communication occurs. The mathematical framework underlying a 3DSSL game can be characterized as a decentralized partially observable MDP (Dec-POMDP) [8]. However, during training, each agent can access all available information, making the widely-adopted centralized-learning-decentralized-execution scheme [7, 19, 38, 56, 57] a desirable option.

3.2 PRACTICAL APPLICATIONS

Gao et al. [16] present an extensive list of skills and other relevant research concerning the 3DSSL. In the scope of this work, the main skill-related contributions can be classified into three categories — locomotion, dribbling, and kicking. For locomotion, most works focus on building a strong analytical foundation based on the Zero Moment Point (ZMP) and a model that abstracts the robot’s dynamics, both historically [20, 24] and in recent contributions [25, 36, 51, 52]. One popular simplification approach is the linear inverted pendulum model (LIPM).

The effectiveness of these models can be improved through various approaches, such as interpolation of keyframes [44, 53, 54, 62] or optimization methods. Optimization can be used to complement analytical models, either in parallel [27, 28, 41, 60] or sequentially [15]. It can be used over primitives extracted from observed trajectories [10] or soccer motion capture data [37], or to learn from scratch, as demonstrated in activities like walking [61] and running [2, 3, 42, 43].

Dribbling is an extension of locomotion, with the added effort of controlling the ball. This skill can be divided into close control dribble [2], involving precise ball manipulation while keeping it near the player’s feet; and speed dribble [34, 35, 46], where the robot executes precise kicks to send the ball a bit further and then swiftly pursues it.

Finally, in the literature, kicking is predominantly categorized into two main forms: as the culmination of locomotion skills (implying that the robot is moving at a non-negligible speed before kicking) [5, 49] or as a skill that involves a slower preparation stage before kicking. In the latter case, contributions vary from attempting to cover the largest distance on a given axis [12, 21, 23], including sideways kicks [13], to customizing the target distance [1, 30] and the kick direction [58]. The static or moving initial conditions represent a trade-off between accuracy and reaction time.

4 SKILL-SET-PRIMITIVES

This section introduces a type of analytical structure that serves as the foundation for learning more complex behaviors. In this work, we propose *skill-set-primitives*, a subclass of *motion primitives* [11, 47] designed for a set of skills instead of a single behavior. This type of primitive captures the commonalities of a set of actions. For instance, while walking, humans lift each foot alternately, regardless of the direction of motion, even during rotations. This underlying movement template can also be applied to marching, running, or dribbling a ball.

Skill-set-primitives are characterized by several key attributes that distinguish them within the realm of control schemes for robotic motion:

- 1-to-many scheme — instead of using primitives as building blocks to develop other skills in a 1-to-1 or many-to-1 scheme [10, 37], we use them as an integrating foundation to develop and unite a specific set of skills;
- Continuity — identifying a persistent pattern, often cyclic, facilitates transitions between elements within the skill set, since they share the same root. In practice, this means that the alternating foot lift primitive mentioned above can operate in the background continuously while a soccer agent seamlessly transitions between walking, running, and dribbling. In addition to providing the skill set with a uniform style, which makes transitions easier, they can also be synchronized through the primitive’s cycle state.
- Reduced bias — In contrast to analytical models that employ comprehensive solutions (e.g., walking engines) or primitives based on fully developed behaviors, skill-set-primitives can adopt a simpler form, as they are not required to mimic functional behaviors; they can even be abstract or unintelligible. Consequently, skill-set-primitives contain less information, leading to a reduced number of error-prone assumptions embedded in behaviors.
- Constant and reliable — primitives can be designed analytically or learned as the first step of Sequential Layered Learning [59]. In both cases, using the many-to-1 scheme requires the resulting model to be frozen and provide reliable outputs for all potential input combinations. Primitives with feedback from the environment (close-loop control) are discouraged in large state spaces since they may contain unexplored areas. These areas may be visited while learning new skills, leading the primitive to produce erratic outputs that can impede the training process.

In addition to promoting skill integration due to shared motion patterns, skill-set-primitives also accelerate development by serving as a logical starting point. Reiterating, the introduced bias is reduced, as primitives do not need to mimic functional behaviors. However, providing the skill set with a uniform style can be exploited to produce symmetric and human-like movements, enhancing the aesthetics and realism of humanoid robot skills.

5 SKILL SETS

Two skill sets have been developed by FC Portugal: the 2021 Sprint-Kick and the more recent locomotion set. An overview of the encompassing framework can be seen in Fig. 2. Each skill is represented by a single neural network policy, except for the locomotion set kick, which has two variants. This section will present the hierarchical structure and execution sequence of both sets, as well as the corresponding skill-set-primitives.

5.1 SPRINT-KICK

The Sprint-Kick development involved three learning stages:

1. **Sprint Forward:** first, the agent learned to sprint forward, with an acceleration phase followed by a cyclic phase, repeating every 14 time steps (0.28 seconds) as the robot maintained an average speed of 3.69 m/s. Average joint positions were then extracted from the cyclic phase, resulting in a skill-set-primitive with 14 hard-coded time steps;

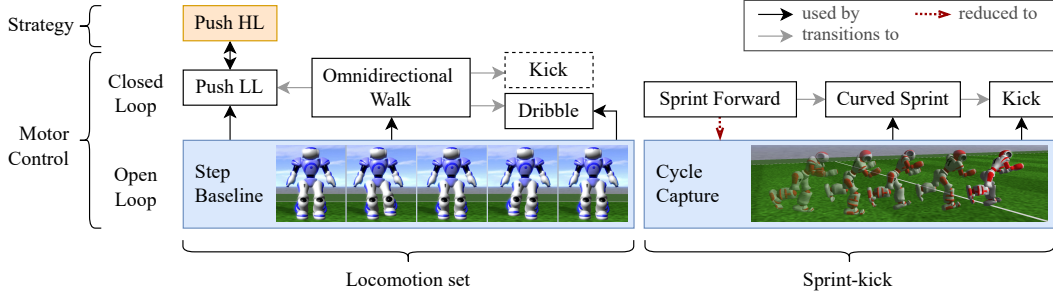


Figure 2: Overview of the robot’s primitive-based motion framework and its hierarchical structure. The lower layer comprises skill-set-primitives (blue boxes) for two skill sets: the current locomotion set used by FC Portugal and the Sprint-Kick used in 2021. These primitives have no feedback from the environment and serve as a foundation to develop complex skills from the second layer (white boxes). The high-level Push (yellow box) is the only behavior trained in a multi-agent environment.

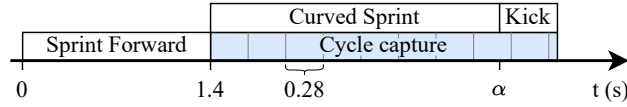


Figure 3: Execution of the Sprint-Kick skill set. The robot sprints forward for 1.4 seconds, then changes direction to pursue the ball, and, when closer than 1 meter (α), it starts the kick stage.

2. **Curved Sprint:** after accelerating, the agent learned residuals to improve the primitive, enabling it to gradually and continually change direction — performing a curved sprint, as defined by Filter et al. [14], at an average speed of 3.44 m/s. The turning rate is a continuous parameter configurable between -33 and 33 degrees per second;
3. **Kick:** within one meter of the ball and in the midst of sprinting, the agent learned to modify the primitive so as to collide with the ball and generate a powerful forward kick.

The sequence of events characterizing the Sprint-Kick is shown in Fig. 3, with α denoting the first moment when the sprinter is within one meter of the ball. The cycle capture primitive provides the default joint positions during the execution of the last two skills. The kick can begin from any point in the gait, as the primitive will ensure a smooth transition.

5.2 LOCOMOTION SET

In Section 4, we noted that the alternating lifting of each foot is a common motion trait in bipedal locomotion activities, even during ball control. To capture this trait, we simplified Kasaei et al.’s [29] walking engine by fixing the model’s center of mass (COM) height c_z and eliminating its capacity for movement in any direction. The resulting primitive is a cyclic and smooth stationary walk, termed *Step Baseline*. The equations governing the COM trajectory in y (c_y), and the swing height (h) for a single step are defined as follows:

$$c_y(t) = p_y \left[1 - \operatorname{csch} \left(\frac{T}{w} \right) \left(\sinh \left(\frac{T-t}{w} \right) + \sinh \left(\frac{t}{w} \right) \right) \right], \quad (9)$$

$$h(t) = H * \sin \left(\pi \cdot \frac{t}{T} \right), \quad (10)$$

with $w = \sqrt{c_z/g}$, where g is the gravity acceleration. T denotes the step duration, t the elapsed time, H the swing range, and p_y the zero moment point’s (ZMP) location in y (corresponding to the position of the support foot). In the final primitive, every parameter was optimized concurrently with the Omnidirectional Walk model to maximize the walk stability and speed. Two parameter schemes

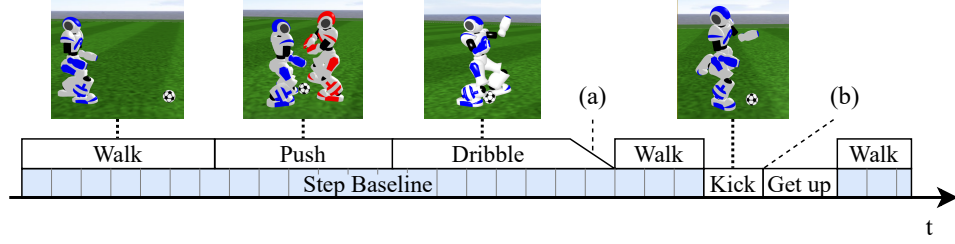


Figure 4: Example of a locomotion set execution sequence. The Step Baseline operates in the background as the agent walks to the ball, pushes it past the opponent, dribbles to an empty space, gradually reverts to the Step Baseline to transition to walking (a), and positions itself for the kick. It then performs a long kick and falls (b), gets up, and resumes walking.

were tested: static and dynamic (i.e., controlled by the walk policy). Ultimately, the static approach exhibited greater stability and allowed for easier transitions.

The skill-set-primitive was then frozen, and all parameters were reused for dribbling and pushing, except for p_y , which was increased to motivate a wider leg stance, except for robot R3 due to its default wide stance. Refer to A.1, Table 8 for the optimized values. Note that c_z is given as a percentage of the COM’s height when the robot is standing with straight legs, since different robot types vary in height and weight distribution. The second dribbling version reverts to the p_y used during walking, as a wider leg stance is more likely to trigger ball-holding fouls under the new rules.

The locomotion set consists of the following skills, ordered chronologically:

1. **Omnidirectional Walk:** the policy directs the robot to the target location while ensuring it aligns with the desired orientation. A single neural network automatically handles acceleration, deceleration, rotation, and walking in any direction;
2. **Kick:** the Kick starts directly from the walking skill when the robot is roughly aligned with the target. The model learned to compensate for small target alignment deviations. It comprises a short kick for precise distances ranging from 3 m to 9 m and a long kick capable of reaching up to 21 m with very low lateral deviation.
3. **Dribble:** dribbling uses the Step Baseline, and transitions directly from walking. The policy combines forward locomotion with rotation, while keeping the ball within close range. The only design distinction in the second version developed for RoboCup 2023 is its expanded ball range to comply with the new ball-holding rules. This adjustment was achieved during training by setting the reward to zero when the ball was within 0.115 meters.
4. **Push:** the Push is comparable to dribbling but is designed for collision-prone scenarios. It comprises two concurrently learned skills. The high-level (HL) part, operating at 3.125 Hz, generates a target push direction to maximize ball possession while moving it to a user-requested location. It takes into consideration the positions of close teammates and opponents. The low-level (LL) part controls the robot joints at 50 Hz to steer the ball toward the HL target.

Fig. 4 presents an execution sequence example, where the Step Baseline is active during walking, pushing, and dribbling. A matrix of skill transitions is provided in A.1, Table 9. Transitions were either *trained* as part of skill optimization, *innate*, if they emerged naturally, or *assisted*, when a gradual return to the Step Baseline is required before walking (as seen in Fig. 4, (a)).

Most skills were trained following a walking phase. The transition from Push to Dribble is innate due to their similarity, especially considering that the former skill is slower. Another instance is when the robot is still standing after kicking and subsequently starts walking.

6 TRAINING ENVIRONMENT

This section describes the most important characteristics of the environments used to train the aforementioned skills. For a comprehensive list of environment characteristics, please refer to A.2 and

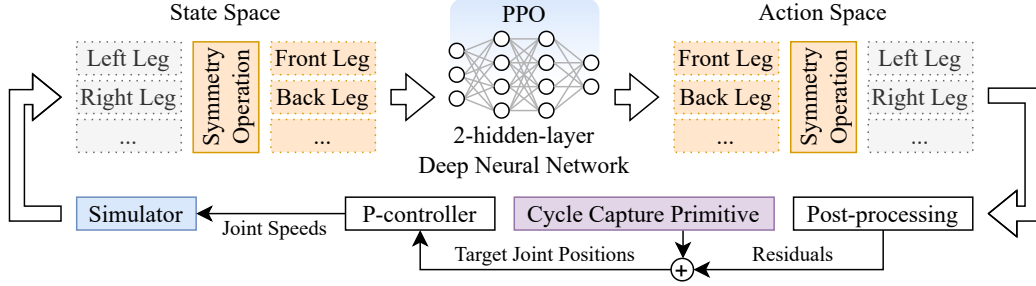


Figure 5: Sprint-Kick training environment. Symmetry operations abstract the RL algorithm from the left and right sides of the robot, forcing the policy to learn a symmetric behavior. The cycle capture primitive is extracted from *Sprint Forward* and added to *Curved Sprint* and *Kick*.

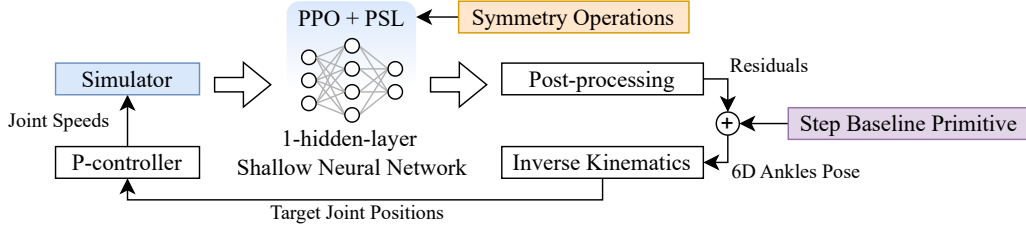


Figure 6: Training environment for the Omnidirectional Walk, Push LL, and Dribble skills, using the Step Baseline primitive. The PPO algorithm is extended with the Proximal Symmetry Loss to allow for a simpler symmetry configuration and reduce human-induced bias.

A.3, which cover details such as state space, symmetry operations (referenced with index/multiplier notation [6]), action space modifications, episode setup, reward functions, and hyperparameters.

6.1 SPRINT-KICK

The Sprint-Kick training environment is outlined in Fig. 5. A 2-hidden-layer deep neural network is trained with PPO and abstracted from the left and right sides of the robot. To do this, we relabel states and actions so that the learning policy is only aware of a front and a back leg. The robot begins sprinting with the left leg in the front, and whenever the right leg takes the lead, all observations and actions are mirrored along the robot’s sagittal plane. The resulting action goes through a post-processing stage including a low-pass filter, and mapping raw joint values to their respective operational ranges. The primitive that captures the *Sprint Forward* motion cycle is then added to the subsequent *Curved Sprint* and *Kick* skills.

6.2 LOCOMOTION SET

Omnidirectional Walk, Push LL, and Dribble are skills from the locomotion set that rely on the Step Baseline primitive. Fig. 6 shows the corresponding training environment. PPO is used in conjunction with Proximal Symmetry Loss (PSL) [6], which transfers the symmetry burden to the neural network, eliminating the need for symmetry operations after training. Moreover, PSL reduces human bias by allowing the policy to explore asymmetric solutions, and it does not impose user-defined symmetry switches.

Instead of generating raw joint values for the legs, the new training environment produces relative 6D poses for both ankles. Then, a low-pass filter with a strong smoothing factor stabilizes the movement. The action values are individually mapped to the desired initial exploration range. Subsequently, the Step Baseline primitive is integrated, and the action is converted into target joint positions using inverse kinematics. These characteristics, associated with a rich state space allow the policy to be simplified into a shallow neural network with a single hidden layer with 64 neurons.

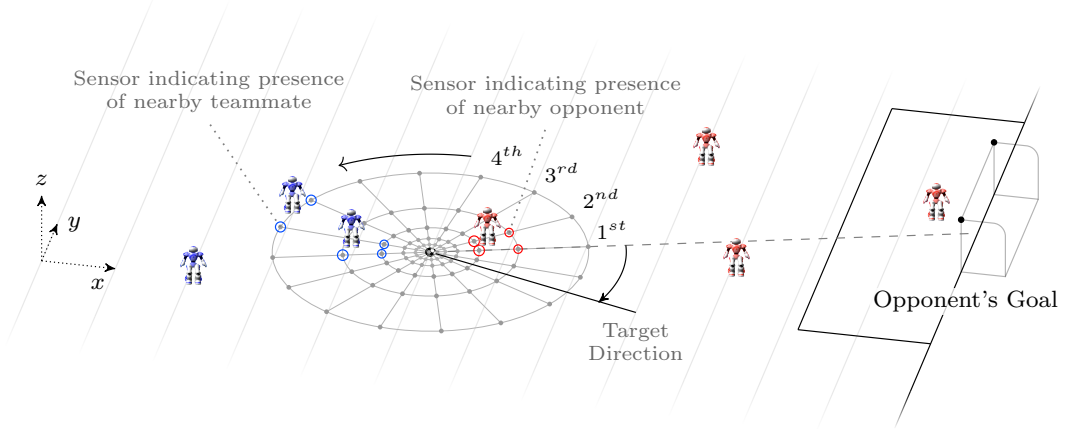


Figure 7: Push HL multi-agent training environment. The policy is aware of teammates and opponents that surround the ball when deciding a short-term target direction.

This modification drastically increases sample efficiency during training, consequently reducing the learning time, while achieving excellent performance, as is demonstrated in Section 7.

6.2.1 PUSH HL

The high-level part of the Push skill is trained in a multi-agent environment, as illustrated in Fig. 7. The neural network receives radar data as input, i.e., the space around the ball is segmented by 5 circles and 16 radial line segments, yielding 80 intersection points. Each point corresponds to the location of two sensors, one for detecting nearby teammates, and another for opponents, totaling 160 independent sensors. For a more interactive explanation of this training environment, please refer to the video demonstration at <https://youtu.be/rGWN83FBdJ4>.

The radar is always oriented toward the user-defined long-term objective direction, which corresponds to the opponent’s goal during training. As shown in Fig. 7, the first sensor of the array always captures the players that are between the ball and the objective. While the order of each sensor is not important, it is imperative to the neural network that each input has a constant meaning, since it will output a short-term target direction, which is given in relation to the direction of the long-term target.

Since this training is very resource-intensive, only the closest player to the ball per team is simulated. The remaining players are represented by a 2D point with a position, velocity, and acceleration, and follow a simplified locomotion model. Despite the speed optimizations, a complete training session takes a full week to run on an Intel Xeon 6258R.

This problem fits into the centralized-learning-decentralized-execution scheme. Although the game is defined as a decentralized partially observable MDP (Dec-POMDP), the learning environment has access to the exact position of each agent and the ball. During actual games, FC Portugal uses the communication channel to address partial observability by attempting to build a common worldview. Consequently, the learning problem can be approximated and solved as a MDP. The policy primarily focuses on the ball, and the episode ends only when the ball is relocated to the center of the field. Meanwhile, the policy observes multiple teammates trying to advance the ball through their Push LL skill, which evolves over time, adding a dynamic component to the environment. The target decision is influenced by all agents that are near the ball, as well as the evolving abilities of the shared Push LL skill.

7 RESULTS AND DISCUSSION

Demonstrations of all skills can be found on YouTube⁵. Major results are summarized in Table 1. The Sprint Forward skill, which is the base of the Sprint-Kick, attains an average speed of 3.69 m/s after stabilizing — an improvement over prior work [3] (2.5 m/s). While Melo et al. [42], reported an average speed of 3.76 m/s under controlled conditions, our sprint was the fastest sprint to be integrated into a team and demonstrated in an official RoboCup competition. The Sprint-Kick sacrifices some speed for maneuverability, being ultimately used to score goals during kickoff set plays. It was also the first kick in the league to be performed while running. For more results, see Appendix A.4.

Table 1: Results per skill

Skill	Metric	Result
Sprint-Kick	Sprint Forward	average speed 3.69 m/s
	Curved Sprint	average speed turning radius 3.44 m/s 6.02 m
	Kick	average distance 9.07 m
Omnidirectional Walk		maximum speed 0.70–0.90 m/s ¹
Kick	Short Kick	avg. error (3/6/9 m) 0.14/0.25/0.62 m
	Long Kick	average error average dist. (x) 1.85 m 19.2 m
Dribble	v1	average fwd. speed turn around offset 1.25–1.41 m/s ² 0.8 m
	v2	average fwd. speed turn around offset 0.71 m/s 0.5 m
Push		maximum speed 0.55 m/s

The Omnidirectional Walk was trained individually for each robot type, offering fast forward walking, rotating while walking in any direction, and precise approaches to kick the ball or wait in strategic positions. Maximum speeds range from 0.70 to 0.90 m/s, depending on walk direction and robot type. Comparing walking skills is challenging due to high variance in versatility. In 2015, Wang et al. [62] achieved 0.5 m/s for forward and backward walking, and 0.4 m/s for side walking, while retaining omnidirectional abilities. In the same year, Shafii et al. [52] optimized a forward walk (0.78 m/s) and a side walk (0.48 m/s). It would be unfair to compare these speeds with an omnidirectional walk. Recent advancements, such as Kasaei et al.’s [27] forward walk with rotation (0.956 m/s) and Fischer et al.’s [15] specialization for robot type 4 (toes) with speeds of 1.3 m/s forward and 1.03 m/s backward, concern specific conditions. However, these approaches lack the all-encompassing control of an omnidirectional walk, with the ability to accelerate, decelerate, rotate, and change walking direction seamlessly and automatically in a single skill. Our behavior is best analyzed in action⁶.

The Short Kick offers variable distance but was tested at 3, 6, and 9 meters, with 1000 samples, to measure distance errors (see Table 1). The Long Kick originated a heatmap with a spatial distribution of final ball positions using robot type 1 (see Fig. 8). Reaching 20 m [12] or more [21] is not uncommon in the league. Moreover, some kicks have special features such as multi-directional targets [58], kick while moving [5, 49], or handling fast-moving balls, which magmaOffenburg mastered for the RoboCup 2023 Kick Rolling Ball Challenge. However, success depends on factors such as kick preparation quality and repeatability, duration, accuracy, and adaptability to in-game disruptions.

⁵<https://www.youtube.com/playlist?list=PLIeNX3I5JIATF1OXfm93d6cijDdRheyyO>
For a comprehensive list of available videos, refer to Appendix A.5.

⁶Omnidirectional Walk training environment and demonstration: <https://youtu.be/dXzIuZlOFZc>

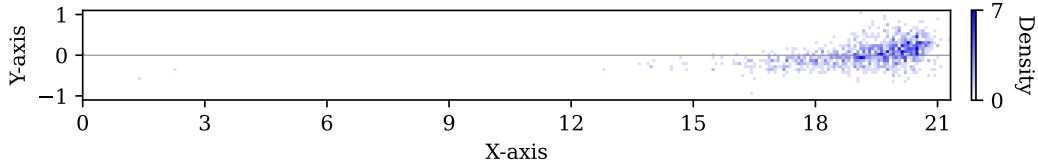


Figure 8: Heatmap of final ball positions after 1000 long kicks using robot type 1

In previous work [2], we introduced a dribble developed from scratch through RL, where the ball is carried between the robot’s legs at 0.31 m/s, which was never integrated into the team for being slow. Dribble v1 was the first close control dribble to be introduced in the 3D Soccer Simulation League, reaching 1.25 to 1.41 m/s, depending on the robot type. The second version complies with new rules by keeping the ball further than 12 cm, albeit at the cost of speed⁷. Other dribble behaviors are not comparable [34, 35, 46], as they focus on kicking the ball further and rapidly pursuing it, rather than maintaining close control throughout.

The Push behavior, designed for ball possession and stability in collision-prone game scenarios, reaches 0.55 m/s. In the 3D league, applying machine learning to high-level strategies remains uncommon due to the complexity of the humanoid soccer multi-agent environment. Related prior work includes Muzio et al.’s dribble technique [45], which trained a higher-level policy for controlling a walking engine by providing translation and rotation commands every 5th step. Despite comprising two layers, the more abstract layer does not address strategy concerns.

In contrast to Austin Villa [39], our focus was on producing fewer behaviors to simplify maintenance. Our proposed methodology effectively handles the added complexity of developing more versatile skills. The most effective evaluation of these skills and their seamless integration is through RoboCup competitions, where FC Portugal has achieved excellent results.

8 LOCOMOTION PERFORMANCE IN ROBOCUP

In the previous section, we presented results for individual skills tested in controlled conditions. While kicks are relatively straightforward to compare with published data and are often used similarly in training and competition, locomotion is heavily influenced by team strategies and must adapt to unforeseen circumstances during competitions. Moreover, comparing locomotion with published data is challenging due to the high variance in target versatility and performance evaluation metrics. To address these challenges, this section presents a statistical analysis of locomotion performance in past RoboCup competitions.

8.1 METHODOLOGY

As highlighted in Section 1, the NAO robot has been the standard platform in RoboCup Soccer since 2008. Consequently, our analysis begins with data from 2008, encompassing all major RoboCup competitions through 2023. Match data were sourced from the *RoboCup.info* online archive [18], and we included all rounds of each competition, including seeding rounds. Penalty shootouts, Fat-Proxy matches, challenges, and test games were excluded. For each match, only data from the PlayOn segments was analyzed, as these periods impose fewer restrictions on player movement.

To ensure consistent and reliable data, we analyzed robot movement using 1-second measurement windows, which mitigates the impact of erratic movements or collisions. Specific exclusion criteria were applied to ensure the validity of these samples:

- **Abnormal height:** Samples were excluded if the robot’s z-axis height was below 0.25 m or above 0.40 m.
- **Excessive speed:** Samples were excluded if the robot’s speed between consecutive time steps exceeded 4 m/s, a threshold slightly higher than the maximum running speed.

⁷Dribble evolution, training environment, and demonstration: https://youtu.be/8UED_Z1-nbQ

Exclusion zones were extended 1 second prior to an invalid time step to account for events like falling, which could affect speed calculations. For example, if a robot begins to fall but is beamed by the referee before its height drops below 0.25 m — due to a play mode change or foul — the fall itself may go undetected, yet the initial falling motion would still be incorrectly factored into speed calculations.

8.2 RESULTS

Table 2 summarizes the average and maximum speeds of RoboCup teams since 2008, aggregated by competition. Teams active since 2021 are ordered by the number of participations, with winners highlighted in red⁸. The key metrics were obtained as follows:

- **Average speed:** computed by aggregating valid 1-second window samples for all teammates in a match, summing the distances covered, and dividing by the total window durations. This method is preferred over averaging individual speeds, as it gives more weight to players with more valid samples, preventing those who frequently fell or crashed from skewing the results. The process is repeated across all games and averaged per team.
- **Maximum speed:** determined by selecting the 5th highest speed from all valid samples across all teammates in a match, ensuring the value reflects sustained performance rather than statistical anomalies.

From preliminary analysis, team speed appears positively correlated with competition success. This observation will be explored in greater detail later in this section.

Table 3 reports the percentage of time robots were fallen and the average distance covered per player, aggregated by team since 2008. The *fallen time* metric reflects the proportion of `PlayOn` time during which a robot’s height was below 0.25 m. Average distances were calculated similarly to average speeds but normalized by the number of players per team, accounting for variations in team sizes over the years.

Note that *total distance* and *average speed* are not redundant metrics because the proportion of valid samples per match varies according to different factors. Consequently, a team may achieve a high average speed while covering a low total distance if its players fall frequently or matches are interrupted often.

Since the soccer field expansion in 2012, FC Portugal had prioritized locomotion stability over speed. In 2019, we introduced the league’s first running behaviors [3], including a sprint at 2.62 m/s with a slow turning rate of 10°/s and a more agile run peaking at 1.52 m/s and 160°/s [4]. However, faster behaviors proved harder to control, as the team’s strategy and path planning were not adapted to the drastic changes, leading to increased collisions, fallen time, and overall instability. In 2021, the team introduced the Sprint-Kick (Section 5.1), stabilizing at 3.44 m/s with a maximum turning rate of 33°/s. The fallen time did not increase beyond 11% because the skill was only used during kickoffs.

Curiously, teams that introduced running behaviors experienced a reduction in the average speed. For FC Portugal (2019) and UT Austin Villa (2023), this was due to instability. In contrast, magmaOffenburg (2022) maintained stability, suggesting their reduced speed and distance were due to deliberate strategy changes.

In 2022, we introduced our new codebase with a dribbling skill peaking at 1.43 m/s. The fallen time was reduced from 11% to 3%, despite a significant increase in covered distance. Maximum walking speed improved from 0.88 m/s to 0.93 m/s, combined with unmatched omnidirectional capabilities (see Section 8.2.2). Then, in 2023, the fast dribbling skill was removed, as explained in Section 1.

8.2.1 PERFORMANCE CORRELATION

To analyze the significance of the previous metrics on team performance, we calculated the Spearman rank correlation coefficients between these metrics and team rankings across all competitions

⁸For completeness, SEU-RedSun won in both 2008 and 2009. In 2008, their average and maximum speeds were 0.12 m/s and 0.61 m/s, with a 15% fallen time and 41 m of average distance per player. In 2009, these values improved to 0.16 m/s, 0.73 m/s, 15%, and 56 m, respectively.

Table 2: Average speed (m/s) and maximum speed (m/s) aggregated by team since 2008. Teams active since 2021 are ordered by number of participations; winners highlighted in red.

	FC Portugal	magma	Offenburg	UT Austin	Villa	BahiaRT	HfutEngine3D	ITAndroids	Apollo3D	Miracle3D	KgpKubs	WITS FC	WrightOcean	MIRG
2008	.03 0.28	-	.02 0.17	-	.07 0.44	-	.11 0.81	-	-	-	-	-	-	-
2009	.09 0.76	.05 0.26	-	.02 0.10	.15 0.65	-	-	-	-	-	-	-	-	-
2010	.19 0.84	.14 0.46	.12 0.75	.03 0.37	.18 0.60	-	.23 0.82	-	-	-	-	-	-	-
2011	.25 0.83	.16 0.67	.26 0.74	.05 0.54	.17 0.73	-	.28 0.86	-	-	-	-	-	-	-
2012	.25 0.84	.20 0.73	.25 0.77	-	-	-	.33 0.93	-	-	-	-	-	-	-
2013	.35 0.73	.28 0.87	.30 0.95	.11 0.45	.20 0.72	.19 0.71	.37 0.94	-	-	-	-	-	-	-
2014	.38 0.78	.35 0.86	.35 0.97	.31 0.86	.17 0.92	-	-	-	-	-	-	-	-	-
2015	.38 0.85	.35 0.85	.38 0.97	.36 0.77	.16 0.92	.26 0.63	.38 0.95	.33 0.61	-	-	-	-	-	-
2016	.39 0.87	.37 0.85	.44 0.97	.34 0.74	.20 0.86	.27 0.76	-	.35 0.61	.28 0.67	-	-	-	-	-
2017	.32 0.88	.39 0.86	.45 0.98	.39 0.87	.36 0.92	.31 0.70	-	.20 0.84	.23 0.68	-	-	-	-	-
2018	.36 0.88	.38 1.05	.41 0.97	.39 0.85	-	.31 0.70	-	.21 0.78	.24 0.67	-	-	-	-	-
2019	.34 2.62	.42 1.06	.44 0.97	.41 0.84	.33 0.83	.34 0.70	-	-	-	-	-	.43 0.89	-	-
2021	.38 3.44	.44 1.07	.47 0.98	.37 0.85	.41 0.91	.33 0.70	.42 1.27	.41 0.79	.26 0.73	.27 0.78	.47 0.90	.27 0.78	-	-
2022	.48 1.43	.39 2.46	.47 0.98	.37 0.95	.40 0.92	.33 0.71	.43 0.82	.40 0.79	.32 0.84	.29 0.78	-	-	-	-
2023	.46 0.93	.39 2.55	.31 1.64	.38 0.88	-	.34 0.70	-	-	-	.28 0.83	-	-	-	-

Table 3: Fallen time (%) and average distance per player (m) aggregated by team since 2008. Teams active since 2021 are ordered by number of participations; winners highlighted in red.

	FC Portugal	magma	Offenburg	UT Austin	Villa	BahiaRT	HfutEngine3D	ITAndroids	Apollo3D	Miracle3D	KgpKubs	WITS FC	WrightOcean	MIRG
2008	10% 13	-	22% 8	-	27% 25	-	14% 41	-	-	-	-	-	-	-
2009	32% 27	6% 21	-	27% 7	13% 52	-	-	-	-	-	-	-	-	-
2010	9% 88	12% 63	6% 65	33% 11	8% 86	-	11% 111	-	-	-	-	-	-	-
2011	11% 113	5% 80	5% 122	27% 15	8% 76	-	8% 128	-	-	-	-	-	-	-
2012	10% 111	3% 101	10% 114	-	-	-	21% 124	-	-	-	-	-	-	-
2013	8% 159	6% 136	4% 143	9% 48	5% 96	14% 75	7% 171	-	-	-	-	-	-	-
2014	8% 193	5% 155	6% 144	7% 169	8% 67	-	-	-	-	-	-	-	-	-
2015	6% 169	4% 168	3% 172	4% 163	9% 65	13% 100	9% 185	19% 118	-	-	-	-	-	-
2016	5% 174	3% 180	3% 193	4% 168	9% 83	6% 123	-	10% 155	4% 139	-	-	-	-	-
2017	5% 156	3% 199	3% 194	6% 185	14% 129	6% 154	-	4% 97	4% 119	-	-	-	-	-
2018	6% 168	4% 185	3% 186	6% 181	-	7% 129	-	4% 99	4% 120	-	-	-	-	-
2019	10% 125	4% 190	3% 184	7% 177	5% 147	5% 140	-	-	-	-	-	4% 161	-	-
2021	11% 134	3% 187	2% 195	6% 160	3% 159	4% 139	3% 180	4% 187	6% 120	3% 111	4% 166	4% 111	-	-
2022	3% 216	4% 164	3% 207	7% 150	3% 172	3% 133	2% 186	3% 179	6% 137	7% 99	-	-	-	-
2023	3% 205	3% 139	6% 86	6% 148	-	4% 140	-	-	-	6% 102	-	-	-	-

(see Table 4). Team rankings, provided in Appendix A, indicate relative positions without reflecting the magnitude of performance differences. Unlike Pearson’s correlation, which assumes linear relationships, we used Spearman’s rank correlation to assess the strength and direction of the monotonic relationship between variables based on their ranks. Note that in competitions before 2015, some teams share the same rank. For example, in 2008, there are 7 rank groups, with the last five teams (ranks 17-21) all assigned rank 7 for the correlation calculation.

The average correlation coefficients across all competitions were 0.81 for average speed, 0.60 for maximum speed, 0.81 for covered distance, and -0.49 for fallen time. As expected, the maximum speed is a weaker predictor of performance compared to average speed or covered distance. Notably, in 2019 and 2021, FC Portugal’s maximum speed was significantly higher than other teams, yet it ranked 6th in both competitions.

The fallen time metric shows a negative correlation, indicating that greater fallen times, which suggest lower stability, are associated with lower rankings. While the magnitude of this correlation is weaker compared to other metrics, it still points to a meaningful relationship. The 2012 competition, however, was an outlier, where this trend was inverted, with teams exhibiting higher fallen times ranking better.

8.2.2 DIRECTIONAL ANALYSIS

In RoboCup competitions, robots mostly walk forward; however, walking backward or sideways is sometimes required, such as when aligning for a shot. To better characterize how teams employ omnidirectional locomotion, we examined how players move in 8 directions: forward, backward, sideways left and right, and the intermediate directions.

Table 4: Spearman rank correlation coefficients between team rankings and their performance metrics across years

	2008	2009	2010	2011	2012	2013	2014	2015	2016	2017	2018	2019	2021	2022	2023
Avg. Sp.	0.81	0.72	0.86	0.86	0.70	0.90	0.79	0.79	0.87	0.75	0.79	0.82	0.92	0.79	0.77
Max. Sp.	0.74	0.58	0.69	0.60	0.38	0.62	0.74	0.52	0.50	0.39	0.86	0.43	0.76	0.61	0.66
Distance	0.80	0.89	0.88	0.89	0.88	0.89	0.65	0.80	0.88	0.85	0.86	0.89	0.85	0.78	0.31
Fallen T.	-0.27	-0.32	-0.41	-0.46	0.21	-0.29	-0.62	-0.77	-0.50	-0.53	-0.61	-0.82	-0.54	-0.62	-0.83

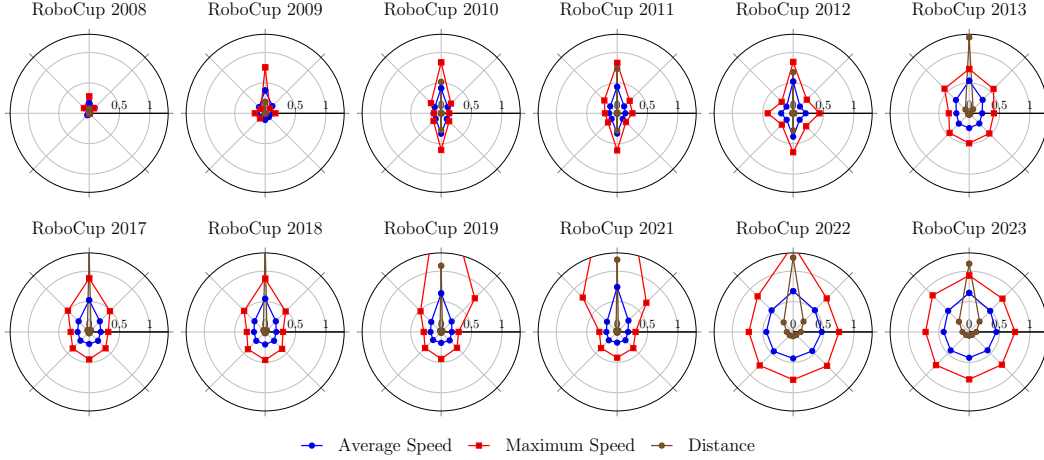


Figure 9: FC Portugal locomotion statistics for periods of notable change (2008–2013, 2017–2023). Metrics include average speed (m/s), maximum speed (m/s), and distance covered (hm), plotted radially to show variation by movement direction, relative to the robot. Values at the top represent forward movement, values on the left indicate sideways movement to the left, and so on.

Measurements were taken by analyzing 1-second windows where speeds exceeded 0.1 m/s — below this threshold, intentional direction is hard to determine. Displacement vectors were compared to the robot’s average orientation during each window to assign one of 8 movement directions.

Fig. 9 shows directional metrics for FC Portugal’s locomotion during periods of significant evolution (2008–2013, 2017–2023): average speed (m/s), maximum speed (m/s), and distance covered (hm). The radial plots represent a top view of the robot, with the robot’s forward direction pointing upward. Data points are positioned radially to represent the direction of movement: forward at the top, backward at the bottom, leftward at the left, and so on.

Initially, the team focused on developing a forward-walking skill, gradually introducing omnidirectional capabilities. Faster forward-running skills were introduced in 2019 and 2021. Asymmetries in the maximum speed arise from biases in these skills, particularly when stopping from a running state [3]. In 2022, the maximum forward speed was influenced by the dribbling skill. However, in 2023 that metric was solely dictated by the omnidirectional walk, revealing its consistent performance in all directions. Moreover, the distance metric indicates that omnidirectional walking was scarcely used until 2022, with back-walking between 2010 and 2012 being an exception.

Fig. 10 illustrates the same metrics for the six teams participating in the 2023 competition, arranged in order of their rankings; and Fig. 11 aggregates speed metrics for all teams over the years, including average speed, average maximum speed, and maximum speed (all in m/s).

In 2023, the two fastest teams in forward speed were magmaOffenburg and UT Austin Villa. Their skill asymmetries (Fig. 10) are apparent in replays [18]. Interestingly, these asymmetries are in opposite directions, complementing each other to create a symmetrical pattern in the aggregate plot (Fig. 11).

Although the maximum speed metric in the 3DSSL shows minimal apparent improvement from 2008 to 2018, robots fell significantly more often during earlier competitions, limiting the sustain-

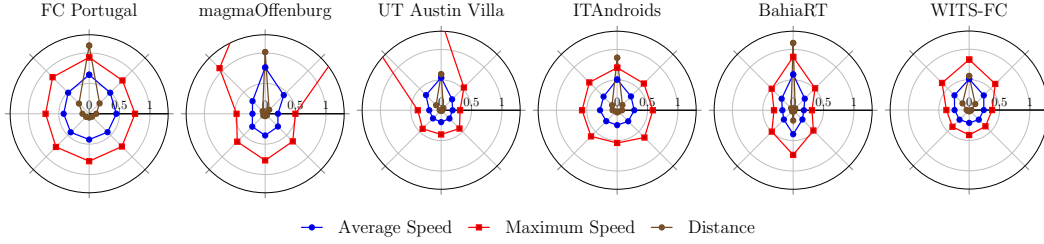


Figure 10: 3DSSL locomotion statistics for the six teams that participated in RoboCup 2023, ordered by competition ranking. Metrics include average speed (m/s), maximum speed (m/s), and distance covered (hm), plotted radially to show variation by movement direction, relative to the robot. Values at the top represent forward movement, values on the left indicate sideways movement to the left, and so on.

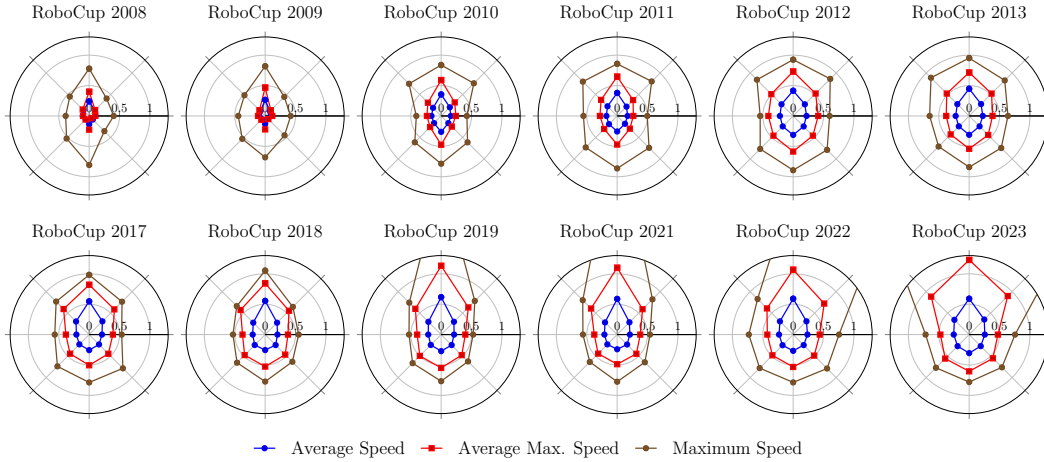


Figure 11: 3DSSL aggregated locomotion statistics for periods of notable change (2008–2013, 2017–2023). Metrics include average speed, average maximum speed, and maximum speed (all in m/s), plotted radially to show variation by movement direction, relative to the robot. Values at the top represent forward movement, values on the left indicate sideways movement to the left, and so on.

ability of their top speed and, consequently, their effectiveness. This conclusion becomes apparent when comparing the average speed metric over the same period.

9 CODEBASE RELEASE

Our codebase release is accessible on GitHub⁹. The project includes a fully functional team of agents that follow a simple formation and attempt to score by kicking the ball toward the opponent’s goal. The primary agent derives from `Base Agent`, a class that provides multiple features, as depicted in Fig. 12. This class incorporates a representation of both the `Robot` and the `World`. The initial representation is obtained from the server and is further enhanced through internal processing, through a `6D Pose Estimator` [5] and a `rolling Ball Predictor`. The agent is equipped with essential tools such as a pathfinding algorithm, inverse kinematics, and extensive data about the robot and its environment. Team communication is facilitated through the `Radio`, automatically sharing information about visible players and the ball. While not all skills are included for competitive reasons, this paper provides all the necessary details to train and replicate the skills mentioned in Table 1. The codebase release contains the `Omnidirectional Walk`, and `Dribble v1`, along with the

⁹FC Portugal Codebase release: <https://github.com/m-abr/FCPCodebase>

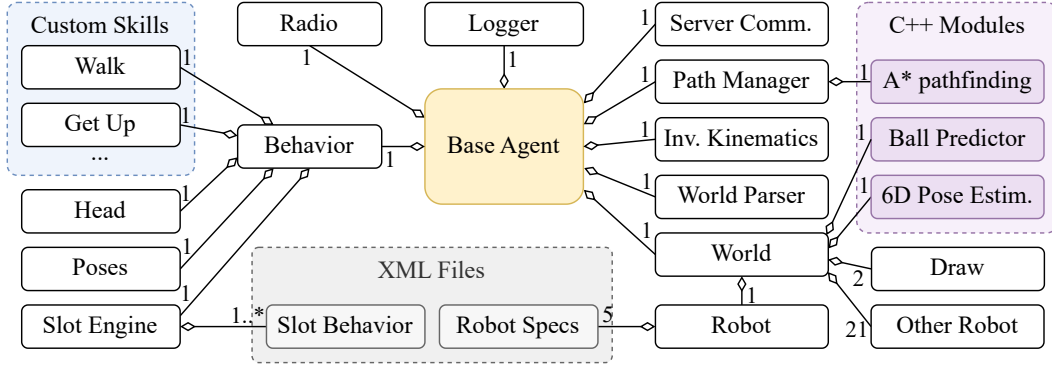


Figure 12: Base Agent structure

underlying Step Baseline skill-set-primitive. Additionally, it includes the latest get-up behaviors, a basic kick, and a basic goalkeeper dive.

10 CONCLUSION

This work introduces the skill set developed for FC Portugal, a team that has proven its capabilities by winning the last two RoboCup 3D Soccer Simulation League World Championships. The methodology used to develop these skills is based on two concepts: a custom algorithm that extends PPO with symmetry-leveraging abilities and skill-set-primitives. These primitives are a subset of motion primitives, and capture common action patterns, making it easier to transition between different behaviors. Furthermore, they allow the policy to be simplified, improving sample efficiency and stability, while still retaining excellent performance in learning complex behaviors. Skill-set-primitives facilitate faster learning without introducing bias, apart from setting the initial exploration area within the solution space.

The first skill created with this framework was the Sprint-Kick, which involves sprinting toward the ball and executing a powerful kick without slowing down. In 2021, this skill was introduced as the fastest sprint to be demonstrated in official league matches (3.69 m/s forward, and 3.44 m/s while curving). It was also the fastest kick.

Following that, we developed the locomotion set, a collection of skills based on one skill-set-primitive, which now forms the core of FC Portugal’s team. This set includes an omnidirectional walk that accelerates, decelerates, rotates, and changes direction seamlessly and automatically, while being controlled by a shallow neural network with a single hidden layer of 64 neurons. The Dribble skill allowed for unprecedented ball control and maneuverability, while the Push skill addressed collision-prone scenarios, increasing ball possession and game stability. We employed virtual agents to streamline the learning process in a multi-agent environment, enabling efficient learning within a reasonable timeframe.

Additionally, we have released the codebase of our team to the robotics community. This open-source approach enables other teams to build upon our work, fostering collaboration and innovation in the realm of robotic soccer.

ACKNOWLEDGMENTS

The first author was supported by the Foundation for Science and Technology (FCT) under grant SFRH/BD/139926/2018. Additionally, this research was financially supported by FCT/MCTES (PIDDAC), under projects UIDB/00027/2020 (LIACC) and UIDB/00127/2020 (IEETA).

REFERENCES

- [1] Abbas Abdolmaleki, David Simões, Nuno Lau, Luis Paulo Reis, and Gerhard Neumann. Learning a humanoid kick with controlled distance. In *RoboCup 2016: Robot World Cup XX 20*, pp. 45–57. Springer, 2017.
- [2] Miguel Abreu, Nuno Lau, Armando Sousa, and Luis Paulo Reis. Learning low level skills from scratch for humanoid robot soccer using deep reinforcement learning. In *2019 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*, pp. 256–263. IEEE, 2019.
- [3] Miguel Abreu, Luis Paulo Reis, and Nuno Lau. Learning to run faster in a humanoid robot soccer environment through reinforcement learning. In *RoboCup 2019: Robot World Cup XXIII*, pp. 3–15. Springer, 2019.
- [4] Miguel Abreu, David Simões, Nuno Lau, and Luís Paulo Reis. Fast, human-like running and sprinting. https://archive.robocup.info/Soccer/Simulation/3D/FCPs/RoboCup/2019/FCPortugal_SS3D_RC2019_FCP.pdf, 2019.
- [5] Miguel Abreu, Tiago Silva, Henrique Teixeira, Luís Paulo Reis, and Nuno Lau. 6D localization and kicking for humanoid robotic soccer. *Journal of Intelligent & Robotic Systems*, 102(2): 1–25, 2021.
- [6] Miguel Abreu, Luís Paulo Reis, and Nuno Lau. Addressing imperfect symmetry: A novel symmetry-learning actor-critic extension. *Neurocomputing*, 614:128771, 2025. ISSN 0925-2312.
- [7] Rana Azzam, Igor Boiko, and Yahya Zweiri. Swarm cooperative navigation using centralized training and decentralized execution. *Drones*, 7(3), 2023.
- [8] Daniel S Bernstein, Robert Givan, Neil Immerman, and Shlomo Zilberstein. The complexity of decentralized control of Markov decision processes. *Mathematics of Operations Research*, 27(4):819–840, 2002.
- [9] Joschka Boedecker and Minoru Asada. SimSpark—concepts and application in the RoboCup 3D soccer simulation league. *Autonomous Robots*, 174:181, 2008.
- [10] Carlos A Acosta Calderon, Rajesh E Mohan, Lingyun Hu, Changjiu Zhou, and Huosheng Hu. Generating human-like soccer primitives from human data. *Robotics and Autonomous Systems*, 57(8):860–869, 2009.
- [11] Joachim Denk and Günther Schmidt. Synthesis of walking primitive databases for biped robots in 3d-environments. In *2003 IEEE international conference on robotics and automation (Cat. No. 03CH37422)*, volume 1, pp. 1343–1349. IEEE, 2003.
- [12] Mike Depinet, Patrick MacAlpine, and Peter Stone. Keyframe sampling, optimization, and behavior integration: Towards long-distance kicking in the RoboCup 3D simulation league. In *RoboCup 2014: Robot World Cup XVIII 18*, pp. 571–582. Springer, 2015.
- [13] Klaus Dorer. Learning to use toes in a humanoid robot. In *RoboCup 2017: Robot World Cup XXI 11*, pp. 168–179. Springer, 2018.
- [14] Alberto Fíler, Jesús Olivares, Alfredo Santalla, Fabio Y Nakamura, Irineu Loturco, and Bernardo Requena. New curve sprint test for soccer players: Reliability and relationship with linear sprint. *Journal of sports sciences*, 38(11-12):1320–1325, 2020.
- [15] Jens Fischer and Klaus Dorer. Learning to walk with toes. In *Artificial Intelligence: Research Impact on Key Industries. Proceedings of the Upper-Rhine Artificial Intelligence Symposium*, pp. 3–11, 2020.
- [16] Zhongye Gao, Mengjun Yi, Ying Jin, Hanwen Zhang, Yun Hao, Ming Yin, Ziwen Cai, and Furao Shen. A survey of research on several problems in the RoboCup3D simulation environment. *Autonomous Agents and Multi-Agent Systems*, 38(1):13, Mar 2024.

- [17] Guillermo Garcia-Hernando, Edward Johns, and Tae-Kyun Kim. Physics-based dexterous manipulations with estimated hand poses and residual reinforcement learning. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 9561–9568. IEEE, 2020.
- [18] Stefan Glaser. robocup.info archive: RoboCup Soccer – 3D Simulation League. <https://archive.robocup.info/Soccer/Simulation/3D/>.
- [19] Jayesh K Gupta, Maxim Egorov, and Mykel Kochenderfer. Cooperative multi-agent control using deep reinforcement learning. In *Autonomous Agents and Multiagent Systems: AAMAS 2017 Workshops, Best Papers, São Paulo, Brazil, May 8-12, 2017, Revised Selected Papers 16*, pp. 66–83. Springer, 2017.
- [20] Kazuo Hirai, Masato Hirose, Yuji Haikawa, and Toru Takenaka. The development of Honda humanoid robot. In *Proceedings. 1998 IEEE international conference on robotics and automation (Cat. No. 98CH36146)*, volume 2, pp. 1321–1326. IEEE, 1998.
- [21] Xinpeng Hu, Zhuolun Li, Guolong Sun, and Baofu Fang. Apply acceleration sampling to learn kick motion for nao humanoid robot. In *2020 International Conference on Computer Engineering and Intelligent Control (ICCEIC)*, pp. 318–323. IEEE, 2020.
- [22] Tobias Johannink, Shikhar Bahl, Ashvin Nair, Jianlan Luo, Avinash Kumar, Matthias Loskyll, Juan Aparicio Ojea, Eugen Solowjow, and Sergey Levine. Residual reinforcement learning for robot control. In *2019 International Conference on Robotics and Automation (ICRA)*, pp. 6023–6029. IEEE, 2019.
- [23] Nicolas Jouandeau and Vincent Hugel. Optimization of parametrised kicking motion for humanoid soccer player. In *2014 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*, pp. 241–246. IEEE, 2014.
- [24] Shuuji Kajita, Fumio Kanehiro, Kenji Kaneko, Kiyoshi Fujiwara, Kensuke Harada, Kazuhito Yokoi, and Hirohisa Hirukawa. Biped walking pattern generation by using preview control of zero-moment point. In *2003 IEEE international conference on robotics and automation (Cat. No. 03CH37422)*, volume 2, pp. 1620–1626. IEEE, 2003.
- [25] Mohammadreza Kasaei, Nuno Lau, and Artur Pereira. A fast and stable omnidirectional walking engine for the NAO humanoid robot. In *RoboCup 2019: Robot World Cup XXIII 23*, pp. 99–111. Springer, 2019.
- [26] Mohammadreza Kasaei, Miguel Abreu, Nuno Lau, Artur Pereira, and Luis Paulo Reis. A CPG-based agile and versatile locomotion framework using proximal symmetry loss. *arXiv preprint arXiv:2103.00928*, 2021.
- [27] Mohammadreza Kasaei, Miguel Abreu, Nuno Lau, Artur Pereira, and Luis Paulo Reis. Robust biped locomotion using deep reinforcement learning on top of an analytical control approach. *Robotics and Autonomous Systems*, 146:103900, 2021.
- [28] Mohammadreza Kasaei, Miguel Abreu, Nuno Lau, Artur Pereira, Luis Paulo Reis, and Zhibin Li. Learning hybrid locomotion skills — learn to exploit residual actions and modulate model-based gait control. *Frontiers in Robotics and AI*, 10:1004490, 2023.
- [29] S Mohammadreza Kasaei, Nuno Lau, Artur Pereira, and Ehsan Shahri. A reliable model-based walking engine with push recovery capability. In *2017 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*, pp. 122–127. IEEE, 2017.
- [30] Atsuto Kawazoe, Nobuhiro Ito, and Kazunori Iwata. A design method for kicks of the desired distance by combinations of based kicks in rss3d. In *2019 6th International Conference on Computational Science/Intelligence and Applied Informatics (CSII)*, pp. 66–71. IEEE, 2019.
- [31] Hiroaki Kitano and Minoru Asada. RoboCup humanoid challenge: That’s one small step for a robot, one giant leap for mankind. In *Proceedings. 1998 IEEE/RSJ international conference on intelligent robots and systems. Innovations in theory, practice and applications (Cat. No. 98CH36190)*, volume 1, pp. 419–424. IEEE, 1998.

- [32] Hiroaki Kitano, Minoru Asada, Yasuo Kuniyoshi, Itsuki Noda, Eiichi Osawa, and Hitoshi Matsubara. RoboCup: a challenge problem for AI. *AI magazine*, 18(1):73–73, 1997.
- [33] Ashish Kumar, Zhongyu Li, Jun Zeng, Deepak Pathak, Koushil Sreenath, and Jitendra Malik. Adapting rapid motor adaptation for bipedal robots. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 1161–1168. IEEE, 2022.
- [34] David L Leottau, Javier Ruiz-del Solar, Patrick MacAlpine, and Peter Stone. A study of layered learning strategies applied to individual behaviors in robot soccer. In *RoboCup 2015: Robot World Cup XIX 19*, pp. 290–302. Springer, 2015.
- [35] Leonardo Leottau, Carlos Celemin, and Javier Ruiz-del Solar. Ball dribbling for humanoid biped robots: a reinforcement learning and fuzzy control approach. In *RoboCup 2014: Robot World Cup XVIII 18*, pp. 549–561. Springer, 2015.
- [36] Zhiwei Liang, Hecheng Zhao, and Yue Hao. An omnidirectional walk for a biped robot based on gyroscope-accelerometer measurement. In *2014 IEEE International Conference on Mechatronics and Automation*, pp. 1052–1057. IEEE, 2014.
- [37] Siqi Liu, Guy Lever, Zhe Wang, Josh Merel, SM Ali Eslami, Daniel Hennes, Wojciech M Czarnecki, Yuval Tassa, Shayegan Omidshafiei, Abbas Abdolmaleki, et al. From motor control to team play in simulated humanoid football. *Science Robotics*, 7(69):eabo0235, 2022.
- [38] Ryan Lowe, Yi I Wu, Aviv Tamar, Jean Harb, OpenAI Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in Neural Information Processing Systems*, 30, 2017.
- [39] Patrick MacAlpine and Peter Stone. Overlapping layered learning. *Artificial Intelligence*, 254: 21–43, 2018.
- [40] Patrick MacAlpine, Mike Depinet, and Peter Stone. UT Austin Villa 2014: RoboCup 3D simulation league champion via overlapping layered learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 29, 2015.
- [41] Dicksiano C Melo, Marcos ROA Maximo, and Adilson Marques da Cunha. Learning push recovery behaviors for humanoid walking using deep reinforcement learning. *Journal of Intelligent & Robotic Systems*, 106(1):8, 2022.
- [42] Luckeciano C Melo, Dicksiano C Melo, and Marcos ROA Maximo. Learning humanoid robot running motions with symmetry incentive through proximal policy optimization. *Journal of Intelligent & Robotic Systems*, 102(3):54, 2021.
- [43] Luckeciano Carvalho Melo and Marcos Ricardo Omena Albuquerque Máximo. Learning humanoid robot running skills through proximal policy optimization. In *2019 Latin american robotics symposium (LARS), 2019 Brazilian symposium on robotics (SBR) and 2019 workshop on robotics in education (WRE)*, pp. 37–42. IEEE, 2019.
- [44] Francisco Muniz, Marcos ROA Maximo, and Carlos HC Ribeiro. Keyframe movement optimization for simulated humanoid robot using a parallel optimization framework. In *2016 XIII Latin American Robotics Symposium and IV Brazilian Robotics Symposium (LARS/SBR)*, pp. 79–84. IEEE, 2016.
- [45] Alexandre FV Muzio, Marcos ROA Maximo, and Takashi Yoneyama. Deep reinforcement learning for humanoid robot dribbling. In *2020 Latin American Robotics Symposium (LARS), 2020 Brazilian Symposium on Robotics (SBR) and 2020 Workshop on Robotics in Education (WRE)*, pp. 1–6. IEEE, 2020.
- [46] Alexandre FV Muzio, Marcos ROA Maximo, and Takashi Yoneyama. Deep reinforcement learning for humanoid robot behaviors. *Journal of Intelligent & Robotic Systems*, 105(1):12, 2022.
- [47] Jun Nakanishi, Jun Morimoto, Gen Endo, Gordon Cheng, Stefan Schaal, and Mitsuo Kawato. Learning from demonstration and adaptation of biped locomotion. *Robotics and autonomous systems*, 47(2-3):79–91, 2004.

- [48] B. Ravindran and A. G. Barto. Symmetries and model minimization in Markov decision processes. Technical report, University of Massachusetts, USA, 2001.
- [49] Amin Rezaeipannah, Parvin Amiri, and Shahram Jafari. Performing the kick during walking for RoboCup 3D soccer simulation league using reinforcement learning algorithm. *International Journal of Social Robotics*, 13:1235–1252, 2021.
- [50] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.
- [51] Andreas Seekircher and Ubbo Visser. An adaptive LIPM-based dynamic walk using model parameter optimization on humanoid robots. *KI-Künstliche Intelligenz*, 30(3-4):233–244, 2016.
- [52] Nima Shafii, Nuno Lau, and Luis Paulo Reis. Learning to walk fast: Optimized hip height movement for simulated and real humanoid robots. *Journal of Intelligent & Robotic Systems*, 80:555–571, 2015.
- [53] Haobin Shi, Xuesi Li, Huahui Chen, and Shixiong Wang. Adaptive omni-directional walking method with fuzzy interpolation for biped robots. *International Journal of Networked and Distributed Computing*, 4(3):145–158, 2016.
- [54] Haobin Shi, Xuesi Li, Weihao Liang, Huahui Chen, Shixiong Wang, et al. A novel fuzzy omni-directional gait planning algorithm for biped robot. In *2016 17th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*, pp. 71–76. IEEE, 2016.
- [55] Tom Silver, Kelsey Allen, Josh Tenenbaum, and Leslie Kaelbling. Residual policy learning. *arXiv preprint arXiv:1812.06298*, 2018.
- [56] David Simões, Nuno Lau, and Luís Paulo Reis. Multi-agent actor centralized-critic with communication. *Neurocomputing*, 390:40–56, 2020.
- [57] Andries Smit, Herman A Engelbrecht, Willie Brink, and Arnau Pretorius. Scaling multi-agent reinforcement learning to full 11 versus 11 simulated robotic football. *Autonomous Agents and Multi-Agent Systems*, 37(1):20, 2023.
- [58] Martin Spitznagel, David Weiler, and Klaus Dorer. Deep reinforcement multi-directional kick-learning of a simulated robot with toes. In *2021 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*, pp. 104–110. IEEE, 2021.
- [59] Peter Stone. *Layered Learning in Multiagent Systems: A Winning Approach to Robotic Soccer*. The MIT Press, Cambridge, MA, 03 2000.
- [60] Chongben Tao, Jie Xue, Zufeng Zhang, Feng Cao, Chunguang Li, and Hanwen Gao. Gait optimization method for humanoid robots based on parallel comprehensive learning particle swarm optimizer algorithm. *Frontiers in Neurobotics*, 14:600885, 2021.
- [61] Chongben Tao, Jie Xue, Zufeng Zhang, and Zhen Gao. Parallel deep reinforcement learning method for gait control of biped robot. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 69(6):2802–2806, 2022.
- [62] Shixiong Wang, Mengkai Hu, Haobin Shi, Shuge Zhang, Xuesi Li, and Wenyan Li. Humanoid robot’s omnidirectional walking. In *2015 IEEE International Conference on Information and Automation*, pp. 381–385, 2015.
- [63] Shimon Whiteson and Peter Stone. Concurrent layered learning. In *Proceedings of the second international joint conference on Autonomous agents and multiagent systems*, pp. 193–200, 2003.
- [64] Yuan Xu and Hedayat Vatankhah. SimSpark: an open source robot simulator developed by the RoboCup community. In *RoboCup 2013: Robot World Cup XVII*, pp. 632–639. Springer, 2014.

- [65] Donghoon Youm, Hyunyoung Jung, Hyeongjun Kim, Jemin Hwangbo, Hae-Won Park, and Sehoon Ha. Imitating and finetuning model predictive control for robust and symmetric quadrupedal locomotion. *IEEE Robotics and Automation Letters*, 8(11):7799–7806, 2023.
- [66] Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative multi-agent games. *Advances in Neural Information Processing Systems*, 35:24611–24624, 2022.
- [67] Andy Zeng, Shuran Song, Johnny Lee, Alberto Rodriguez, and Thomas Funkhouser. Tossing-Bot: learning to throw arbitrary objects with residual physics. *IEEE Transactions on Robotics*, 36(4):1307–1319, 2020.
- [68] Kaiqing Zhang, Zhuoran Yang, and Tamer Başar. Multi-agent reinforcement learning: a selective overview of theories and algorithms. *Handbook of reinforcement learning and control*, pp. 321–384, 2021.

APPENDIX A

This appendix provides additional information about team rankings, configuration parameters for the Step Baseline and skill transitions, and all the parameters and training conditions required to replicate the Sprint-Kick and the skills within the Locomotion Set. Finally, it presents additional results for the Sprint-Kick and a compilation of videos referenced throughout the paper.

Table 5: Team rankings in the RoboCup 3D Soccer Simulation League competition across the years 2008–2012. Teams sharing the same ranking are shown without a separating line.

Rank	2008	2009	2010	2011	2012
1	SEU-RedSun	SEU-RedSun	Apollo3D	UT Austin Villa	UT Austin Villa
2	WrightEagle	Bold Hearts	Nao Team Humboldt	CIT 3D	RoboCanes
3	BATS	AmoiensisNQ	HfutEngine3D	Apollo3D	Bold Hearts
4	CZU 3D	HfutEngine3D	Bold Hearts	KylinSky3D	magmaOffenburg
5	Apollo3D	FUT-K	SEU-RedSun	Bold Hearts	Apollo3D
6	Borregos3D	opuCI 3D	FC Portugal	FC Portugal	ODENS
7	Fantasia	JMU 3D	RoboCanes	RoboCanes	CIT 3D
8	MRL	FC Portugal	DreamWing3D	SEU-RedSun	FC Portugal
9	AmoiensisNQ	Borregos3D	opuCI 3D	OxBlue3D	Karachi Koalas
10	Parsian	magmaOffenburg	magmaOffenburg	beeStanbul	beeStanbul
11	NAITO-Horizon	BahiaRT	FUT-K	magmaOffenburg	FUT-K
12	OxBlue3D	NomoFC	beeStanbul	DreamWing3D	SOCIO 3D
13	FC Portugal	MRL	BATS	Nexus 3D	L3M-SIM
14	HfutEngine3D	Scorpius	UT Austin Villa	HfutEngine3D	
15	NomoFC	RoboPUB	Nexus 3D	FUT-K	
16	opuhana	BATS	Parsian	Karachi Koalas	
17	JMU 3D	BMP	BahiaRT	Nao Team Humboldt	
18	Scorpius	Parsian	NomoFC	Rail	
19	PAT	Persia	RoboPUB	L3M-SIM	
20	Strive3D	Nexus 3D	Alzahra	Farzanegan	
21	UT Austin Villa	NAITO-StrikerS		NomoFC	
22				BahiaRT	

Table 6: Team rankings in the RoboCup 3D Soccer Simulation League competition across the years 2013–2017. Teams sharing the same ranking are shown without a separating line.

Rank	2013	2014	2015	2016	2017
1	Apollo3D	UT Austin Villa	UT Austin Villa	UT Austin Villa	UT Austin Villa
2	UT Austin Villa	RoboCanes	FUT-K	FUT-K	magmaOffenburg
3	FC Portugal	magmaOffenburg	FC Portugal	FC Portugal	FUT-K
4	SEU Jolly	FC Portugal	BahiaRT	BahiaRT	RoboCanes
5	Karachi Koalas	FUT-K	Apollo3D	magmaOffenburg	AIUT3D
6	RoboCanes	BahiaRT	magmaOffenburg	ITAndroids	BahiaRT
7	magmaOffenburg	SEU Jolly	RoboCanes	Miracle3D	FC Portugal
8	Bold Hearts	HfutEngine3D	Nexus 3D	KgpKubs	KgpKubs
9	PHOTON	Karachi Koalas	CIT 3D	HfutEngine3D	ITAndroids
10	HfutEngine3D	L3M-SIM	HfutEngine3D		Miracle3D
11	ITAndroids	ODENS	ITAndroids		Nexus 3D
12	Paydar 3D	Mithras3D	Miracle3D		HfutEngine3D
13	FUT-K				RIC-AASTMT
14	Mithras3D				
15	ODENS				
16	L3M-SIM				
17	BahiaRT				

Table 7: Team rankings in the RoboCup 3D Soccer Simulation League competition across the years 2018–2023

Rank	2018	2019	2021	2022	2023
1	UT Austin Villa	UT Austin Villa	UT Austin Villa	FC Portugal	FC Portugal
2	magmaOffenburg	magmaOffenburg	magmaOffenburg	magmaOffenburg	magmaOffenburg
3	FC Portugal	WrightOcean	Apollo3D	UT Austin Villa	UT Austin Villa
4	BahiaRT	BahiaRT	WrightOcean	Apollo3D	ITAndroids
5	KgpKubs	HfutEngine3D	HfutEngine3D	HfutEngine3D	BahiaRT
6	Miracle3D	FC Portugal	FC Portugal	Miracle3D	WITS FC
7	ITAndroids	ITAndroids	Miracle3D	ITAndroids	
8			ITAndroids	KgpKubs	
9			BahiaRT	WITS FC	
10			KgpKubs	BahiaRT	
11			WITS FC		
12			MIRG		

A.1 STEP BASELINE AND TRANSITIONS

Table 8: Step Baseline Parameters

Parameter	Walk	Dribble (v1)	Push	Dribble (v2)
p_y/F_y	1.12	1.20 ¹	1.20 ¹	1.12
H (m)	0.02			
T (s)	0.16			
c_z (%)	70			

Table 9: Skill transitions

First \ Second	Get Up	Walk	Kick	Dribble	Push
Get Up	-	Trained	No	No	No
Walk	No	-	Trained	Trained	Trained
Kick	No	Innate	-	No	No
Dribble	No	Assisted	No	-	No
Push	No	Assisted	No	Innate	-

A.2 SPRINT-KICK

Table 10: Sprint-Kick state space: *Sprint Forward*, *Sprint & Rotate*, *Kick*

Param. name	Data size $\times 32b$ FP	Description	Symmetry Indices	Symmetry Multiplier
step	1	Step counter ¹ [3]	0	1
count	1	Counter since last side switch	0	1
z	1	Head height prediction [3]	0	1
gyro	3	Gyroscope (x-forward, y-left, z-up)	(0,1,2)	(-1,1,-1)
dgyro	3	$\frac{d}{dt} [\text{gyro}_x \text{ gyro}_y \text{ gyro}_z]$ for $\Delta t = 0.02$ s	(0,1,2)	(-1,1,-1)
acc	3	Accelerometer (x-forward, y-left, z-up)	(0,1,2)	(1,-1,1)
dacc	3	$\frac{d}{dt} [\text{acc}_x \text{ acc}_y \text{ acc}_z]$ for $\Delta t = 0.02$ s	(0,1,2)	(1,-1,1)
feet	12	(l)eft and (r)ight feet sensors with (p)oint of origin and (f)orce vector: (lpx,lpy,lpz,lfx,lfy,l fz, rpx,rpy,rpz,rfx,rfy,rfz)	(6,7,8,9,10,11, 0,1,2,3,4,5)	(1,-1,1,1,-1,1, 1,-1,1,1,-1,1)
dfeet	12	$\frac{d}{dt} [\text{feet}_0 \dots \text{feet}_{11}]$ for $\Delta t = 0.02$ s	(6,7,8,9,10,11, 0,1,2,3,4,5)	(1,-1,1,1,-1,1, 1,-1,1,1,-1,1)
joints	20	All joint positions excluding head and toes	Sagittal reflection ²	Sagittal reflection ²
djoints	20	$\frac{d}{dt} [\text{joints}_0 \dots \text{joints}_{19}]$ for $\Delta t = 0.02$ s	Sagittal reflection ²	Sagittal reflection ²
target ³	1	Target direction relative to the robot’s torso	0	-1
ball ⁴	3	Ball position relative to the robot’s torso (x-forward, y-left, z-up)	(0,1,2)	(1,-1,1)

Table 11: Sprint-Kick action space modification sequence: *Sprint Forward*, *Sprint & Rotate*, *Kick*

Stage	Description
1. Action	Raw target positions for 22 joints (includes toes, excludes head joints)
2. Symmetry	The positions are reflected with respect to the robot’s sagittal plane when the left leg is positioned as the front leg (see Section 6.1)
3. IIR Filter	An exponential moving average filter is applied with a smoothing factor of 0.5
4. Map	The raw values are mapped to the position ranges of each individual joint
5a. Bias	The initial joint positions are added as a permanent bias (only for <i>Sprint Forward</i>)
5b. Primitive	Add cycle capture primitive (only for <i>Sprint & Rotate</i> and <i>Kick</i>)
6. Conversion	First, predict current joint positions by combining the last action with previous positions (the server has a 1-step delay). Then, determine joint speeds through proportional control.

Table 12: Sprint-Kick RL conditions: *Sprint Forward*

Section	Description
Reset	The robot is reset to a fixed initial pose with bent knees, which is then added as a continuous bias (see Table 11, 5a)
Termination	Robot has fallen ($z < 0.27$) or timeout (4 s)
Episode	There is no specific configuration during the episode
Reward	$r = \frac{dx}{dt}$ for $\Delta t = 0.06$ s. There is no penalty for deviations in y . The reward is updated every vision cycle.

Table 13: Sprint-Kick RL conditions: *Sprint & Rotate*

Section	Description
Reset	<i>Sprint Forward</i> is executed for 1.4 s (this parameter was found to be the minimal period for which the resulting behavior, <i>Sprint & Rotate</i> , achieves maximum speed)
Termination	Robot has fallen ($z < 0.34$) or timeout (7 s)
Episode	A new target direction is randomly generated every 0.28 s (cycle caption period, see Fig. 3). The target’s range is $[-10, 10]$ deg, relative to the current orientation of the robot’s torso.
Reward	$r = \text{speed_2d} \times (1 - \text{direction_error}/10)$, with the error given in degrees. The reward is updated every vision cycle.

Table 14: Sprint-Kick RL conditions: *Kick*

Section	Description
Reset	The ball is placed in a random but reachable position. <i>Sprint Forward</i> is executed for 1.4 s. Then, <i>Sprint & Rotate</i> is executed until the ball is within 1 meter.
Termination	Robot has fallen ($z < 0.33$) or timeout (0.4 s)
Episode	There is no specific configuration during the episode
Reward	$r = \text{direction_unit_vector} \cdot \text{ball_velocity_2d}$, where $\cdot$ denotes the dot vector. The direction vector is determined immediately after reset by evaluating the direction of the ball relative to the robot. The reward is given at the terminal step.

Table 15: Sprint-Kick hyperparam.: *Sprint Forward* (SF), *Sprint & Rotate* (SR), *Kick* (K)

Parameter	Value	Parameter	Value	Parameter	Value
batch steps (K)	256	γ (gamma)	0.99	NN policy arch.	[64,64]
batch steps (SF, SR)	1024	learning rate (LR)	3×10^{-4}	NN value arch.	[64,64]
clip range	0.2	LR scheduler (SF)	constant	total time steps (K)	40M
entropy coefficient	0	LR scheduler (SR, K)	linear	total time steps (SF)	200M
environments	16	mini-batch size	64	total time steps (SR)	100M
epochs	10	NN activation	ReLU	value coefficient	0.5
GAE λ (lambda)	0.95				

A.3 LOCOMOTION SET

Table 16: *Omnidirectional Walk* state space

Param. name	Data size ×32b FP	Description	Symmetry Indices	Symmetry Multiplier
step	1	Step counter (max: 120) ¹ [3]	0	1
z	1	Head height ²	0	1
dz	1	$\frac{dz}{dt}$ for $\Delta t = 0.04$ s	0	1
roll	1	Torso roll angle ²	0	-1
pitch	1	Torso pitch angle ²	0	1
gyro	3	Gyroscope (x-forward, y-left, z-up)	(0,1,2)	(-1,1,-1)
acc	3	Accelerometer (x-forward, y-left, z-up)	(0,1,2)	(1,-1,1)
feet	12	(l)eft and (r)ight foot sensors with (p)oint of origin and (f)orce vector: (lpx,lpy,lpz,lfx,lfy,lfz,rx,ry,rz)	(6,7,8,9,10,11, 0,1,2,3,4,5)	(1,-1,1,1,-1,1, 1,-1,1,1,-1,1)
ankles	6	(l)eft and (r)ight ankle position relative to the robot's hip: (lx,ly,lz,rx,ry,rz)	(3,4,5,0,1,2)	(1,-1,1,1,-1,1)
feetori	6	(l)eft and (r)ight foot orientation relative to the robot's hip: (lx,ly,lz,rx,ry,rz)	(3,4,5,0,1,2)	(-1,1,-1,-1,1,-1)
shldrs	4	(l)eft and (r)ight shoulder joints positions	Sagittal reflection ³	Sagittal reflection ³
action	16	Action array generated in last time step	Sagittal reflection ³	Sagittal reflection ³
sbprog	1	Step Baseline cycle progress $\in [0, 1]$	0	1
sbleg	2	One-hot encoding for the current Step Baseline swing leg (left leg, right leg)	(1,0)	(1,1)
targp	2	Filtered ⁴ 2D target position vector, relative to the robot's torso (x-forward, y-left)	(0,1)	(1,-1)
targv	2	$\frac{d}{dt} [\text{targp}_x \text{ targp}_y]$ for $\Delta t = 0.02$ s	(0,1)	(1,-1)
targdir	1	Filtered ⁴ target direction, relative to the robot's torso	0	-1

Table 17: *Omnidirectional Walk* action space modification sequence

Stage	Description
1. Action	Relative 6D pose of ankles relative to hip (12 values), shoulder joint positions (4 values)
2. IIR Filter	An exponential moving average filter is applied with a smoothing factor of 0.2
3. Global scale	The filtered action is multiplied by $\ \text{targp}\ _2 \times 3.5 + 0.3$ if $\ \text{targp}\ _2 < 0.2$ to encourage the robot to minimize movement as it approaches the target position
4. Map	The individual values are mapped to the desired initial exploration range
5. Primitive	Add Step Baseline primitive
6. Constraints	Limit leg joints to prevent self-collisions
7. IK	Apply inverse kinematics to obtain the target joint positions
8. Conversion	First, predict current joint positions by combining the last action with previous positions (the server has a 1-step delay). Then, determine joint speeds through proportional control.

Table 18: *Omnidirectional Walk* RL conditions

Section	Description
Reset	Execute Get Up behavior
Termination	Robot has fallen ($z < 0.35$)
Episode	Two targets are created: position (TP) and direction (TD). A random velocity is periodically assigned (avg. interval of 1.6 s) to a moving TP within a work area of 196 m ² . Upon leaving the work area, the TP is relocated to its center. In 30% of episodes, TD coincides with TP, whereas in 70% of episodes, TD changes randomly at an avg. interval of 4 s.
Reward	$r = \text{progress} \times \text{direction_penalty} + \text{idle_bonus}$, where progress represents the distance reduction in relation to TP, $\text{direction_penalty} = 1.03^{-\text{direction_error}}$ is a penalty for deviating direction_error degrees from TD, and idle_bonus is an incentive for precision, rewarding short movements when the target is within 0.2 m. The reward is updated every vision cycle.

Table 19: *Omnidirectional Walk* hyperparameters

Parameter	Value	Parameter	Value	Parameter	Value
batch steps	2048	γ (gamma)	0.99	NN value arch.	[64]
clip range	0.2	learning rate (LR)	3×10^{-4}	PSL value weight	0.5
entropy coefficient	0	LR scheduler	constant	PSL policy weight	0.005
environments	32	mini-batch size	64	total time steps	50M
epochs	10	NN activation	ReLU	value coefficient	0.5
GAE λ (lambda)	0.95	NN policy arch.	[64]		

Table 20: Kick state space: *Short Kick* and *Long Kick*

Param. name	Data size $\times 32\text{b FP}$	Description
step	1	Step counter ¹ [3]
z	1	Head height ²
dz	1	$\frac{dz}{dt}$ for $\Delta t = 0.04$ s
roll	1	Torso roll angle ²
pitch	1	Torso pitch angle ²
gyro	3	Gyroscope (x-forward, y-left, z-up)
acc	3	Accelerometer (x-forward, y-left, z-up)
feet	12	(l)eft and (r)ight feet sensors with (p)oint of origin and (f)orce vector: (lpx,lpy,lpz,lfx,lfy,lfz,rxp,rpy,rpz,rfx,rfy,r fz)
joints	16	All joint positions excluding head, toes and 4 elbow joints
djoints	16	$\frac{d}{dt} [\text{joints}_0 \dots \text{joints}_{15}]$ for $\Delta t = 0.02$ s
ballp	3	Ball position relative to the robot’s hip (x-forward, y-left, z-up)
ballv	3	$\frac{d}{dt} [\text{ball}_x \text{ ball}_y \text{ ball}_z]$ for $\Delta t = 0.04$ s
balld	1	Euclidean distance from ball to robot’s hip
targdir	1	Target direction, relative to the robot’s torso
targd ³	1	Target distance

Table 21: Kick action space modification sequence: *Short Kick* and *Long Kick*

Stage	Description
1. Action	Raw target speed for all joints, excluding head, toes and 4 elbow joints (16 values)
2. Map	The individual values are mapped to the desired initial exploration range
3. Bias	An initial bias is added to pull the kicking leg back for 0.04 s (for <i>Short Kick</i>) or 0.12 s (for <i>Long Kick</i>) to generate a back swing

Table 22: Kick RL conditions: *Short Kick* and *Long Kick*

Section	Description
Reset	The target direction is constant. The robot is initially placed near the ball with both a random position and orientation. The ball is given a random velocity with a magnitude inferior to 1 m/s. Then, the robot aligns itself with the ball and the target as it approaches. The alignment is stricter for the <i>Long Kick</i> . Additionally, for the <i>Short Kick</i> , the target distance is randomly set between 3 m and 9 m.
Termination	Timeout (0.20 s for <i>Short Kick</i> and 0.32 s for <i>Long Kick</i>)
Episode	There is no specific configuration during the episode
Reward	$r = \text{initial_ball_targ_dist} - \text{final_ball_targ_dist}$, where the final distance between ball and target is measured after the episode terminates, when the ball speed drops below 0.2 m/s. The reward waits for the ball to stop but is assigned to the terminal step.

Table 23: Kick hyperparameters: *Short Kick* (SK) and *Long Kick* (LK)

Parameter	Value	Parameter	Value	Parameter	Value
batch steps (LK)	128	GAE λ (lambda)	0.95	NN activation	ReLU
batch steps (SK)	120	γ (gamma)	0.99	NN policy arch.	[64]
clip range	0.2	learning rate (LR)	3×10^{-4}	NN value arch.	[64]
entropy coefficient	0	LR scheduler	constant	total time steps (LK)	25M
environments	24	mini-batch size (LK)	64	total time steps (SK)	15M
epochs	10	mini-batch size (SK)	60	value coefficient	0.5

Table 24: *Dribble* state space

Param. name	Data size $\times 32\text{b FP}$	Description	Symmetry Indices	Symmetry Multiplier
step	1	Step counter (max: 96) ¹ [3]	0	1
z	1	Head height ²	0	1
dz	1	$\frac{dz}{dt}$ for $\Delta t = 0.04$ s	0	1
roll	1	Torso roll angle ²	0	-1
pitch	1	Torso pitch angle ²	0	1
gyro	3	Gyroscope (x-forward, y-left, z-up)	(0,1,2)	(-1,1,-1)
acc	3	Accelerometer (x-forward, y-left, z-up)	(0,1,2)	(1,-1,1)
feet	12	(l)eft and (r)ight feet sensors with (p)oint of origin and (f)orce vector: (lpx,lpy,lpz,lfx,lfy,l fz, rpx,rpy,rpz,rfx,r fy,rfz)	(6,7,8,9,10,11, 0,1,2,3,4,5)	(1,-1,1,1,-1,1, 1,-1,1,1,-1,1)
joints	20	All joint positions excluding head and toes	Sagittal reflection ³	Sagittal reflection ³
djoints	20	$\frac{d}{dt} [\text{joints}_0 \dots \text{joints}_{19}]$ for $\Delta t = 0.02$ s	Sagittal reflection ³	Sagittal reflection ³
sbprog	1	Step Baseline cycle progress $\in [0, 1]$	0	1
sbleg	2	One-hot encoding for the current Step Baseline swing leg (left leg, right leg)	(1,0)	(1,1)
sbsin	1	Step Baseline oscillation: $\sin(\text{sbprog} \times \pi)$	0	1
ballp	3	Ball position relative to the robot's hip (x-forward, y-left, z-up)	(0,1,2)	(1,-1,1)
ballv	3	$\frac{d}{dt} [\text{ball}_x \text{ ball}_y \text{ ball}_z]$ for $\Delta t = 0.04$ s	(0,1,2)	(1,-1,1)
balld	1	Euclidean distance from ball to robot's hip	0	1
targdir	1	Filtered ⁴ target direction, relative to the robot's torso	0	-1
targv	1	$\frac{dtargdir}{dt}$ for $\Delta t = 0.02$ s	0	-1

Table 25: *Dribble* action space modification sequence

Stage	Description
1. Action	Relative 6D pose of ankles relative to hip (12 values), shoulder joint positions (4 values)
2. IIR Filter	An exponential moving average filter is applied with a smoothing factor of 0.15
3. Global scale	A global scaling parameter allows the robot to gradually slow down and revert to the Step Baseline primitive for an easy transition to walking
4. Map	The individual values are mapped to the desired initial exploration range
5. Primitive	Add Step Baseline primitive
6. Constraints	Limit leg joints to prevent self-collisions
7. IK	Apply inverse kinematics to obtain the target joint positions
8. Conversion	First, predict current joint positions by combining the last action with previous positions (the server has a 1-step delay). Then, determine joint speeds through proportional control.

Table 26: *Dribble* conditions

Section	Description
Reset	The robot is initially placed near the ball with both a random position and orientation. The ball is given a random velocity with a magnitude inferior to 1 m/s. Then, the robot aligns itself with the ball and starts the approach until it is close enough (< 0.25 m).
Termination	Robot has fallen ($z < 0.40$), timeout (40 s), ball was lost, or ball cannot be seen
Episode	A new target direction is randomly generated on average every 0.33 s
Reward	$r = \text{ball_speed} \times \cos(\text{direction_error})$, where direction_error is the deviation, in radians, between the ball velocity vector and the filtered target direction vector used as observation. In the second dribble version, the reward is set to zero when the ball is within 0.115 meters to avoid triggering the new ball-holding foul. The reward is updated every vision cycle.

Table 27: *Dribble* hyperparameters

Parameter	Value	Parameter	Value	Parameter	Value
batch steps	2048	γ (gamma)	0.99	NN value arch.	[64]
clip range	0.2	learning rate (LR)	3×10^{-4}	PSL value weight	0.5
entropy coefficient	0	LR scheduler	constant	PSL policy weight	0.002
environments	32	mini-batch size	64	total time steps	150M
epochs	10	NN activation	ReLU	value coefficient	0.5
GAE λ (lambda)	0.95	NN policy arch.	[64]		

Table 28: *Push LL* state space

Param. name	Data size ×32b FP	Description	Symmetry Indices	Symmetry Multiplier
step	1	Step counter (max: 96) ¹ [3]	0	1
z	1	Head height ²	0	1
dz	1	$\frac{dz}{dt}$ for $\Delta t = 0.04$ s	0	1
roll	1	Torso roll angle ²	0	-1
pitch	1	Torso pitch angle ²	0	1
gyro	3	Gyroscope (x-forward, y-left, z-up)	(0,1,2)	(-1,1,-1)
acc	3	Accelerometer (x-forward, y-left, z-up)	(0,1,2)	(1,-1,1)
feet	12	(l)eft and (r)ight feet sensors with (p)oint of origin and (f)orce vector: (lpx,lpy,lpz,lfx,lfy,lfz,rpx,rpy,rpz,rfx,rfy,rfz)	(6,7,8,9,10,11, 0,1,2,3,4,5)	(1,-1,1,1,-1,1, 1,-1,1,1,-1,1)
joints	20	All joint positions excluding head and toes	Sagittal reflection ³	Sagittal reflection ³
djoints	20	$\frac{d}{dt} [\text{joints}_0 \dots \text{joints}_{19}]$ for $\Delta t = 0.02$ s	Sagittal reflection ³	Sagittal reflection ³
sbprog	1	Step Baseline cycle progress $\in [0, 1]$	0	1
sbleg	2	One-hot encoding for the current Step Baseline swing leg (left leg, right leg)	(1,0)	(1,1)
sbsin	1	Step Baseline oscillation: $\sin(\text{sbprog} \times \pi)$	0	1
ballp	3	Ball position relative to the robot’s hip (x-forward, y-left, z-up)	(0,1,2)	(1,-1,1)
ballv	3	$\frac{d}{dt} [\text{ball}_x \text{ ball}_y \text{ ball}_z]$ for $\Delta t = 0.04$ s	(0,1,2)	(1,-1,1)
balld	1	Euclidean distance from ball to robot’s hip	0	1
targp	2	2D target position vector, relative to the robot’s torso (x-forward, y-left)	(0,1)	(1,-1)
opp	2	Weighted ⁴ average of closest opponent’s visible limbs positions, relative to our robot’s torso	(0,1)	(1,-1)

Table 29: *Push LL* action space modification sequence

Stage	Description
1. Action	Relative 6D pose of ankles relative to hip (12 values), shoulder joint positions (4 values)
2. IIR Filter	An exponential moving average filter is applied with a smoothing factor of 0.2
3. Map	The individual values are mapped to the desired initial exploration range
4. Primitive	Add Step Baseline primitive
5. Constraints	Limit leg joints to prevent self-collisions
6. IK	Apply inverse kinematics to obtain the target joint positions
7. Conversion	First, predict current joint positions by combining the last action with previous positions (the server has a 1-step delay). Then, determine joint speeds through proportional control.

Table 30: *Push LL* conditions

Section	Description
Reset	One robot from our team approaches the ball, aligns itself with it, and begins the approach until close enough. If it falls or gets overtaken by a teammate, the active player is switched, and the procedure is restarted.
Termination	Robot has fallen ($z < 0.40$), timeout (8 s), ball was lost, ball cannot be seen, teammate is closer to ball
Episode	The opposing team executes 1 time step: the closest opponent to the ball is physically simulated and uses the Omnidirectional Walk to push the ball to our goal. The other 10 opponents adhere to a simplified locomotion model, use FC Portugal’s formation, and are represented as 2D points with velocity and acceleration. Our team executes an analogous behavior, except for the current closest player, which is the only player controlled by the policy being learned.
Reward	$r = \min(\text{ball_speed}, 0.5) \times \max(0, \cos(\text{direction_error}))^7$, where <i>direction_error</i> is the deviation, in radians, between the ball velocity vector and the target position vector generated by <i>Push HL</i> . The ball speed is limited to 0.5 m/s to prioritize ball possession and progression over high speed. The reward is updated every vision cycle.

Table 31: *Push LL* hyperparameters

Parameter	Value	Parameter	Value	Parameter	Value
batch steps	4096	γ (gamma)	0.99	NN value arch.	[64]
clip range	0.2	learning rate (LR)	3×10^{-4}	PSL value weight	0.5
entropy coefficient	0	LR scheduler	constant	PSL policy weight	0.001
environments	32	mini-batch size	64	total time steps	320M
epochs	10	NN activation	ReLU	value coefficient	0.5
GAE λ (lambda)	0.95	NN policy arch.	[64]		

Table 32: *Push HL* state space

Param. name	Data size $\times 32\text{b FP}$	Description	Symmetry Indices	Symmetry Multiplier
radar	160	Radial spatial segmentation where each segment indicates the presence of opponents and/or teammates (see Section 6.2.1)	Sagittal reflection ¹	Sagittal reflection ¹
mate	1	Closest teammate Euclidean distance	0	1
opp	1	Closest opponent Euclidean distance	0	1
targdir	1	Last HL target direction, relative to user-defined long-term goal direction	0	1

Table 33: *Push HL* action space modification sequence

Stage	Description
1. Action	Target direction (1 value)
2. Map	The direction is mapped to the range [-90,90]
3. Filter	A filtered direction is set to a maximum variation of 30 deg per iteration

Table 34: *Push HL* conditions

Section	Description
Reset	Wait for <i>Push LL</i> to reset
Termination	Push LL resets and, simultaneously, ball exits learning area ($x \in [-12, 12], y \in [-9, 9]$)
Episode	Push LL is executed 16 times prior to re-observing the environment and running the policy, at a frequency of 3.125 Hz.
Reward	$r = \text{clip}(\text{progress} \cdot 0.15, -0.02, 0.02) + \text{clip}(\text{advantage} \cdot 0.4, -0.3, 0.3)$, with $\text{progress} = \text{initial_ball_targ_dist} - \text{final_ball_targ_dist}$, and $\text{advantage} = \text{closest_opponent_dist} - \text{closest_teammate_dist}$. As in <i>Push LL</i> , the objective is to prioritize ball possession and progression over high speed. The reward is updated every 0.32 s.

Table 35: *Push HL* hyperparameters

Parameter	Value	Parameter	Value	Parameter	Value
batch steps	512	γ (gamma)	0.99	NN value arch.	[64,64]
clip range	0.2	learning rate (LR)	3×10^{-4}	PSL value weight	0.5
entropy coefficient	0	LR scheduler	constant	PSL policy weight	0.01
environments	32	mini-batch size	64	total time steps	20M
epochs	10	NN activation	ReLU	value coefficient	0.5
GAE λ (lambda)	0.95	NN policy arch.	[64,64]		

A.4 ADDITIONAL RESULTS

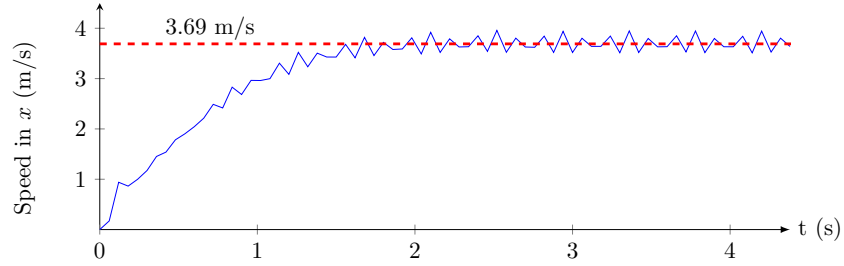


Figure 13: Sprint-Kick: average forward sprint from 5 consecutive samples. The robot stabilizes at an average speed of 3.69 m/s after approximately 2 seconds

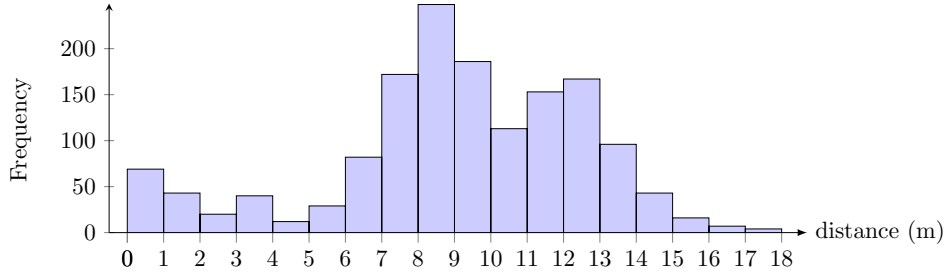


Figure 14: Analysis of 1500 Sprint-Kick samples. The histogram illustrates the distribution of kick distances, i.e., the distance the ball traveled after being kicked. The most frequent range was between 8 and 9 meters.

A.5 LIST OF VIDEOS

Table 36: Demonstration videos

Skill	Video ID ¹	Description
Sprint-Kick	Yy1yCM5hwZI	Sprint-Kick training environment
	3MND8RVUPBQ	Successful Sprint-Kick Kickoffs in RoboCup 2021
	udN2F3oXAec	Sprint-Kick slow motion
Omnidirectional Walk	dXzIuZIOFZc	Omnidirectional Walk training env. and demonstration
Short Kick	trn1KzXFhwc	Short Kick training environment
Long Kick	pu4g5wryfEs	Best kick goals in RoboCup 2022
	QQXGgpcnVYE	Long Kick test to assess the final ball position
Dribble	8UED-Zl-nbQ	Dribble evolution, training environment and demonstration
Push	rGWN83FBdJ4	Push training environment
	mhfQ0NAmlMI	Push highlights from Brazil Open 2022